

无线电干扰源的快速自动定位与清除模型构建及模拟测试

摘要

针对测向误差有界、干扰源位置与接收半径未知，以及定向干扰源可能从背向无法观测等困难，本文建立“几何定位—测点优化—完备搜索—分层清除”的统一模型。首先将每次示向度观测表示为张角为 2° 的方位楔，通过半平面交得到凸多边形可行域；随后以区域直径、最小包围圆半径和任务总时间分别刻画定位精度与清除效率。

对于问题 1，采用半平面裁剪、旋转卡壳和 **Welzl 随机增量算法** 计算定位区域直径与最小包围圆，并证明“以区域直径为直径的圆”不一定覆盖整个定位区域。

对于问题 2，在四端点圆盘交形成的代理候选区内，以四个端点场景的最大定位面积为目标建立极小极大模型，得到第二检测点策略 $S_2^* = S_1 + 928.8e_{\parallel} \pm 382.8e_{\perp}$ 。蒙特卡洛实验验证的 3000 个共享随机场景中，该策略无接收失败。

对于问题 3，以 9 个探测点实现全向源的全域覆盖，并结合交会定位法、最小包围圆定位、开放路径 2-opt 优化及分层细定位完成搜索清除。10 次演练均全部清除，各次“平均每源时间”的算术均值为 421.9 s；3 次正式测试共清除 35 个干扰源，按全部干扰源加权的平均每源时间为 386.53 s，程序运行时间不超过 0.46 s。

对于问题 4，以距离可达探测点的最大方向间隙不超过 180° 为判据构造 28 点三环网格；对目标圆域内 36 万个位置的高密度数值校核所得最大方向间隙为 174.3° 。10 次演练均全部清除，各次“平均每源时间”的算术均值为 651.4 s；3 次正式测试共清除 37 个干扰源，加权平均每源时间为 666.74 s，程序运行时间不超过 3.35 s。

几何推导、端点场景复核、蒙特卡洛试验与官方模拟器测试共同表明，所建模型能够在给定误差和接收半径范围内完成定位与清除。模型用集合几何统一描述测向不确定性，并将检测完备性直接嵌入搜索路径设计；其主要代价是为提高最坏情形下的可靠性，需要承担固定的覆盖扫描时间。

关键字： 交会定位法 极小极大优化 最小包围圆 覆盖扫描 路径规划

一、问题重述

1.1 问题背景

无线电干扰源的定位与清除是无线电频谱管理的重要任务。传统精准定位依赖技术人员徒步排查，效率低且危险。示向度可由阵列测向、空间谱估计等方法获得 [?], 而多站交会定位是无线电侦察与电子对抗中的经典处理思路 [?]. 为此, 某公司拟研发搭载于机器狗的自动定位清除系统。目标区域为半径 1800m 的圆形区域, 干扰源分布于 1–20 频道, 分全向 (360°) 与定向 (定向方向两侧各 90°) 两类。测向机存在 $[-1^\circ, 1^\circ]$ 的示向度误差, 有效接收半径为 1000–1500m。机器狗移动速度为 5 m/s, 检测、频道切换、光学定位与激光清除均需固定时间。本文需要在定位不确定性、覆盖完备性与任务时间之间取得平衡, 建立可直接转化为机器狗指令序列的搜索清除模型。

1.2 任务要求

问题 1 给定若干检测点坐标及某干扰源关于这些检测点的示向度, 用交会定位法计算多边形定位区域直径, 并判断以该直径为直径的圆能否覆盖此定位区域。

问题 2 已知某全向干扰源在一个检测点处测得的示向度, 给出第二个检测点的选择策略及候选区域, 以获得较好的定位效果。

问题 3 目标区域分布 10–16 个全向干扰源, 数量、频道、位置未知。机器狗从原点出发, 初始频道为 1, 制定自动搜索、定位与清除策略, 在确保全部清除的前提下使总时间最短, 并通过模拟器测试验证。

问题 4 在问题 3 基础上加入定向干扰源, 其定向方向未知, 仅在特定角度内辐射信号。设计相应检测与清除策略及算法, 保证全部干扰源被清除, 并测试有效性。

二、问题分析

问题 1 已知若干检测点坐标及某干扰源关于这些检测点的示向度, 示向度存在 $[-1^\circ, 1^\circ]$ 的固定误差。我们从误差的几何含义出发, 将每个检测点的测量结果建模为以该点为顶点、张角为 2° 、中心方向为示向度的扇形区域; 多个扇形相交形成凸多边形定位区域。于是, 直径计算问题转化为计算几何中的半平面交与凸多边形直径求解问题 [?]. 具体地, 利用半平面交构造定位区域 $\mathcal{P}$, 采用旋转卡壳算法在 $O(n)$ 时间内求出直径 D , 并借助 Welzl 随机增量算法求最小包围圆半径 r^* , 最后利用容格定理 [?] 判定以 D 为直径的圆能否覆盖整个定位区域。该问为后续各问的定位误差分析提供了统一的几何工具与算法基础。

问题 2 已知某全向干扰源在一个检测点处测得的示向度，要求给出第二个检测点的选择策略，以获得关于该干扰源较好的定位效果，并给出第二个检测点的候选区域。需要推导出双检测点交会定位区域面积的解析表达式，分析交会角、两点距离及相对方位对定位精度的影响；以最小化最坏情形下的定位区域面积为准则，建立极小极大优化模型，给出第二个检测点的最优位置与候选区域。该问的结论直接指导问题 3 中机器狗检测点的布设，使粗定位阶段在有限检测次数下获得尽可能小的不确定区域。

问题 3 目标区域为半径 1800 米的圆形区域，其中分布 10 ~ 16 个全向干扰源，频道互不相同且位于 1 ~ 20 号频道，初始频道为 1，具体个数、位置、频道与有效接收半径（1000 ~ 1500 米）均未知。机器狗从原点出发，沿直线以 5 米/秒移动，检测时需停下并用 5 秒获得示向度，频道切换耗时 1 秒；距离干扰源不超过 20 米时可光学精确定位（3 秒）并激光清除（2 秒），距离不超过 5 米且位于覆盖角内时可直接清除。要求在确保所有干扰源被清除的前提下，使定位清除总时间最短。可以利用有效接收半径的下界设计覆盖扫描网格（取 950 米为覆盖裕量半径），保证对任意全向干扰源的检测完备性；在此基础上完成交会粗定位、最近邻清除路径规划与两点细定位，并通过模拟器演练测试统计平均定位清除时间等指标，检验策略有效性。

问题 4 目标区域中既有全向干扰源，又有定向干扰源，总数仍为 10 ~ 16 个，但具体个数未知，定向干扰源的个数及定向方向也未知。定向干扰源仅在定向方向两侧各 90° 范围内辐射信号，范围之外无信号，因此机器狗从背面接近时可能检测不到信号。针对定向干扰源只在覆盖角内可检测的特点，以最大方向间隙判据设计三环扫描网格，保证对任意位置、任意朝向的定向干扰源均可被检测；并引入环形探测机制解决背面接近时的信号缺失问题。其余条件与问题 3 相同，仍以最短时间清除全部干扰源为目标，并通过模拟器演练与三次正式测试验证策略与算法的有效性。

这四个问题呈现出清晰的递进关系：问题 1 建立的方位楔交会与直径算法是全篇的几何基础，其误差刻画直接支撑问题 2 的检测点优化设计；问题 2 得到的交会质量与检测点布设规律又为问题 3 中扫描网格和补测方向的选择提供依据；问题 3 形成“全域扫描—多线交会—路径优化—失败补测”的闭环流程；问题 4 则进一步把距离可达性与方向可见性同时纳入覆盖判据，并增加环形探测与蛇形兜底策略，最终构成一套从理论定位到工程执行的完整解决方案。

我们首先用集合几何描述测向误差，使真实干扰源位置始终包含在方位交会集中；其次以直径、最小包围圆半径和交会面积评价定位不确定性，并据此优化第二检测点；再次利用接收半径下界构造全向源覆盖网格，并通过开放路径优化减少移动距离；最后针对定向源增加方向可见性约束，用距离可达探测点的最大方向间隙给出完备检测条件。这样既避免仅追求平均性能而遗漏极端位置，也使后续模拟器结果能够对应到前述几何判据与时间模型。

三、模型假设

为简化问题、突出主要矛盾，本文在建模过程中做出以下假设：

假设 1：测向机的单次示向误差只需满足有界条件，即真实方位与测得方位之差属于 $[-1^\circ, 1^\circ]$ 。主体模型不额外假设误差服从正态分布或均匀分布；仅在蒙特卡洛验证中，为形成可重复的随机样本而对误差场景作均匀抽样。

假设 2：干扰源在一次任务期间保持静止，其频道、发射功率、有效接收半径与定向方向不随时间变化。全向源在有效接收半径内均可被检测；定向源还需满足检测点落在其 180° 辐射扇区内。该假设与问题 4 的方向可见性约束保持一致。

假设 3：机器狗在两指令位置间沿直线匀速行进，速度为 5 m/s ；检测、频道切换、光学精确定位与激光清除的耗时均按题目给定值计入总时间，从而可将整个任务统一写成移动时间与离散动作时间之和。

假设 4：测向机、机器狗与干扰源均投影到同一水平面内进行二维建模；光学精确定位及清除是否成功只取决于机器狗与干扰源之间的距离是否满足清除阈值，与定向源的辐射方向无关。

四、符号说明

为避免同一字母在不同小问中承担不同含义，全文对主要全局符号统一如下；仅在局部推导中出现的临时下标或中间变量在相应公式后说明。

表 1 主要符号说明

符号	含义
S_i	第 i 个测向检测点
P_i	搜索阶段预设的第 i 个扫描/探测点
G	干扰源真实位置
$\hat{G}, \tilde{G}$	分别表示粗定位估计位置与两点细定位位置
θ_i, θ_i^*	分别为 S_i 处测得示向度与真实方位角
ε	示向度误差上界，主体问题中 $\varepsilon = 1^\circ = \pi/180$
$W(S_i)$ 或 $\mathcal{W}_i$	检测点 S_i （或 P_i ）对应的方位楔可行域
$\mathcal{P}$	多个方位楔相交得到的交会定位多边形
D 或 $D_{\mathcal{P}}$	定位多边形直径
r^*	定位多边形的最小包围圆半径
ρ	以最小包围圆圆心作为估计位置时的最坏位置误差界
d_1, d_2	干扰源到两个检测点的距离

表 1 主要符号说明（续）

符号	含义
φ	两条真实示向线的交会角
A	双检测点交会定位区域面积
α	问题 2 中真实方位相对第一次测得示向轴的偏角
t, L	第二检测点沿示向轴、垂直示向轴的偏移量
$\mathbf{e}_{\parallel}, \mathbf{e}_{\perp}$	分别为第一次测得示向方向及其垂直方向的单位向量
Ω_0	问题 2 中第二检测点的端点约束候选区域
R	干扰源实际有效接收半径, $R \in [1000, 1500]$ m
$R_{\min}$	有效接收半径下界, 取 1000 m
R_{arena}	目标圆域半径, 取 1800 m
R_{scan}	全向源覆盖扫描的裕量半径, 取 950 m
Ω_{arena}	半径 R_{arena} 的目标搜索区域
v	机器狗移动速度, $v = 5$ m/s
π	机器狗的完整动作序列
$N_{\text{mea}}, N_{\text{sw}}$	检测次数与频道切换次数
$N_{\text{ok}}, N_{\text{fail}}$	成功清除次数与失败清除尝试次数
$T_{\text{total}}, T_{\text{avg}}$	定位清除总时间与平均每源时间
N, N_c	区域内干扰源总数与已清除干扰源数
P_{clr}	清除比例, $P_{\text{clr}} = N_c/N$
b	两点细定位的基线长度, 本文取 25 m
γ	定向干扰源的定向方向角
$\mathcal{S}_R(G)$	对位置 G 而言, 在 $R_{\min}$ 内距离可达的探测点集合
$\alpha_{\text{gap}}(G)$	$\mathcal{S}_R(G)$ 中各探测方向在圆周上的最大方向间隙

五、模型的建立与求解

5.1 问题 1：基于交会定位法的定位区域直径求解与覆盖判定

5.1.1 示向度扇形模型

设检测点 $S_i = (x_i, y_i)$ 处测得的示向度为 θ_i ，误差上界为 ε 。则干扰源真实方位角 θ_i^* 满足

$$\theta_i^* \in [\theta_i - \varepsilon, \theta_i + \varepsilon]. \quad (1)$$

其中，方位角定义为 x 轴正向逆时针旋转至检测点指向干扰源方向的夹角。于是干扰源真实位置 G 必位于以 S_i 为顶点、张角为 2ε 、中心方向为 θ_i 的扇形区域 $W(S_i)$ 内，即

$$W(S_i) = \{P : \angle(S_i P) \in [\theta_i - \varepsilon, \theta_i + \varepsilon]\}. \quad (2)$$

其中， $\angle(S_i P)$ 表示向量 $P - S_i$ 的方位角。将扇形边界线方向单位向量记为 $u_1 = (\cos(\theta_i - \varepsilon), \sin(\theta_i - \varepsilon))$ ， $u_2 = (\cos(\theta_i + \varepsilon), \sin(\theta_i + \varepsilon))$ ，则扇形区域可用两个半平面约束等价表示，即

$$W(S_i) = \{P : u_1 \times (P - S_i) \geq 0, u_2 \times (P - S_i) \leq 0\}. \quad (3)$$

其中， $\times$ 表示二维向量叉积的数值，即 $u \times v = u_x v_y - u_y v_x$ 。该表示将每个检测点的测量信息转化为两个线性不等式，便于统一处理。

5.1.2 交会定位区域

当存在 k 个检测点时，干扰源真实位置必须同时满足全部扇形约束，故定位区域为各扇形之交

$$\mathcal{P} = \bigcap_{i=1}^k W(S_i). \quad (4)$$

由于半平面的交集仍为凸集， $\mathcal{P}$ 是凸多边形（在无界情形下为凸多面区域）。当 $k \geq 2$ 且示向线不全部平行时， $\mathcal{P}$ 有界，其顶点由各半平面边界线的交点构成。求 $\mathcal{P}$ 可采用逐边裁剪算法 [?]: 以初始矩形依次被 $2k$ 个半平面裁剪，每次裁剪保留半平面内侧部分。记第 j 个半平面为 $a_j x + b_j y \leq c_j$ ，裁剪后顶点集合为 $V^{(j)}$ ，则

$$V^{(j)} = \text{Clip} \left(V^{(j-1)}, a_j x + b_j y \leq c_j \right). \quad (5)$$

其中， Clip 按照半平面约束式 (??) 的内侧判定对每条边保留合法顶点，并计算边与裁剪线的交点。裁剪完成后得到顶点序列 $V = \{v_1, v_2, \dots, v_n\}$ ， $n \leq 2k$ 。由于示向度误差有界，真实位置恒满足 $G \in \mathcal{P}$ ，这是算法正确性的检验依据。

5.1.3 多边形直径的旋转卡壳算法

定位区域直径定义为区域内任意两点间距离的最大值

$$D = \max_{P, Q \in \mathcal{P}} \|P - Q\|_2. \quad (6)$$

对凸多边形而言，直径必在顶点处取得，即

$$D = \max_{1 \leq i < j \leq n} \|v_i - v_j\|_2. \quad (7)$$

为在 $O(n)$ 时间内求 D ，采用旋转卡壳算法 [?]: 设顶点按逆时针排列，对每条边 $e_i = v_i v_{i+1}$ ，其对跖点为到该边所在直线距离最大的顶点。沿多边形边界旋转两条平行支撑线，仅需 $O(n)$ 次比较即可遍历全部对跖点对。直径对 (v_p, v_q) 即为使式 (??) 取得最大值的顶点对。

5.1.4 最小包围圆与覆盖判定

“集合直径为 D ”只说明任意两点距离不超过 D ，并不推出存在半径 $D/2$ 的圆覆盖集合。反例是边长为 D 的正三角形：其最小覆盖圆半径为 $D/\sqrt{3} > D/2$ 。

最小包围圆是包含全部顶点的半径最小圆

$$r^* = \min_{c, r} \{r : \|v_i - c\|_2 \leq r, i = 1, \dots, n\}. \quad (8)$$

采用 Welzl 随机增量算法 [?] 求解，其思想是按随机顺序逐点维护最小包围圆，新点若在当前圆外则必位于新圆边界上，递归地以边界点集约束求解，期望时间复杂度为 $O(n)$ 。以定位区域直径 D 为直径的圆（半径 $D/2$ ，圆心为某直径对 midpoint m ）能否覆盖区域，即判断

$$\max_{1 \leq i \leq n} \|v_i - m\|_2 \leq D/2. \quad (9)$$

是否成立。由容格定理 [?]，平面上直径为 D 的任意点集的最小包围圆半径满足

$$D/2 \leq r^* \leq D/\sqrt{3}. \quad (10)$$

且 $r^* = D/\sqrt{3}$ 由等边三角形取到。因此当 $r^* > D/2$ 时直径圆必然不能覆盖定位区域；当 $r^* = D/2$ 且直径对 midpoint 恰为最优圆心时能够覆盖。判定准则为：逐一检验各直径对 (v_p, v_q) ，若存在某直径对使全部顶点到其距离不超过 $D/2$ ，则直径圆覆盖成立，否则不成立，此时以最小包围圆作为覆盖定位区域的最小圆。

5.1.5 样例数据检测

为检验半平面裁剪、旋转卡壳、最小包围圆及直径圆覆盖判定的正确性，我们构造三组具有不同几何特征的示向交会数据。各检测站坐标、实测方位角及误差上界见表 ??。

表 2 三组构造样例的检测站与示向度数据

样例	检测站	x_i/m	y_i/m	$\theta_i/(\circ)$	$\varepsilon/(\circ)$
1	S_1	-10	-5	32.5918	4
1	S_2	12	-4	151.4350	4
1	S_3	8	12	235.3402	4
1	S_4	-8	10	316.6366	4
2	S_1	-14	-4	10.6197	3
2	S_2	-10	3	341.5651	3
2	S_3	15	2	192.9946	3
3	S_1	-12	-7	30.2564	8
3	S_2	12	-7	149.7436	8
3	S_3	0	14	270.0000	8

计算时以一个足够大的矩形作为初始区域，并依次使用各扇形对应的 $2k$ 个半平面进行裁剪。随后用旋转卡壳计算直径 D ，并用随机增量法计算最小包围圆半径 r^* 。计算结果列于表 ??。

表 3 三组构造样例的定位区域与覆盖判定结果

样例	顶点数	面积 / m^2	D/m	r^*/m	r^*/D	超出量 / m	直径圆覆盖
1	6	2.238681	2.270242	1.135207	0.500038	0.014015	否
2	4	3.546515	5.015450	2.507725	0.500000	0	是
3	6	13.361118	4.551316	2.459948	0.540492	0.280437	否

图 ?? 给出了三组定位多边形、直径、直径圆与最小包围圆。样例 2 中 $r^* = D/2$ ，直径圆能够覆盖定位区域；样例 1 和样例 3 中 $r^* > D/2$ ，判定为不覆盖。

综上，三组构造样例的数值结果（见表 ??、表 ?? 和图 ??）表明，该方法能够稳定求得定位区域、直径、最小包围圆及覆盖判定结果，其中样例 2 的直径圆可覆盖定位区域，而样例 1 和样例 3 的 $r^* > D/2$ ，直径圆不能覆盖，需以最小包围圆作为覆盖圆。上述模型与算法为后续问题中的定位精度评估和覆盖分析提供了有效基础。

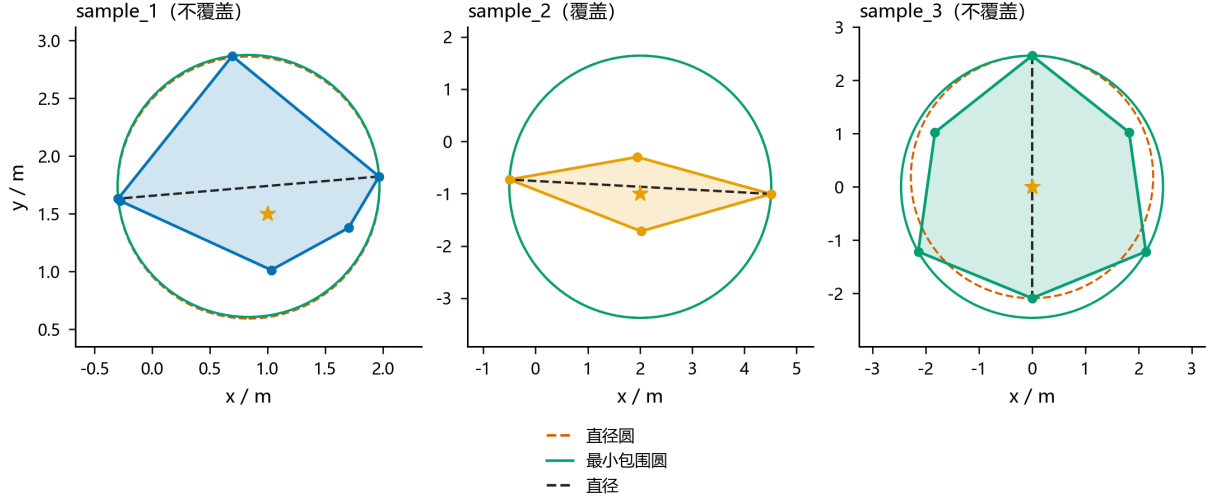


图 1 样例数据最小包围圆与覆盖判定

5.2 问题 2：第二检测点的最优位置选择

5.2.1 建模思路与局部坐标系

第一次测向只能将干扰源限制在一个窄扇形内。第二检测点既要处于所有端点情形的有效接收范围内，又要使两条示向线具有较好的交会角。已有研究常用有界测向楔交集的面积或直径评价布站质量 [?]。最终程序以交会定位面积为指标，并以端点场景的最大面积作为极小极大代理目标。

以第一检测点 S_1 为原点，记第一次测得示向方向的单位向量为 $\mathbf{e}_{\parallel}$ ，其逆时针旋转 90° 后的单位向量为 $\mathbf{e}_{\perp}$ 。在该局部正交坐标系中，沿 $\mathbf{e}_{\parallel}$ 与 $\mathbf{e}_{\perp}$ 的坐标分别记为 t 与 L 。记测向误差上界为

$$\varepsilon = 1^\circ = \frac{\pi}{180} \text{ rad}.$$

若干扰源到 S_1 的距离为 d_1 ，其相对示向轴的偏角为 α ，则不使用目标圆边界信息时，源的局部可能集为

$$W_0 = \{G(d_1, \alpha) = d_1(\cos \alpha, \sin \alpha) : 5 \leq d_1 \leq 1500, |\alpha| \leq \varepsilon\}. \quad (11)$$

其中 5 m 为距离过近阈值，1500 m 为第一次接收所能给出的距离上界。该局部建模不依赖 S_1 的绝对坐标，所得策略可直接随第一次实测示向线旋转和平移。

5.2.2 保证接收的候选区域

为在未知有效接收半径下采取保守策略，程序统一按下界 $R_{\min} = 1000 \text{ m}$ 约束第二次接收。记四个端点场景为

$$G_{d,\sigma} = d(\cos \varepsilon, \sigma \sin \varepsilon), \quad d \in \{5, 1500\}, \quad \sigma \in \{-1, 1\}. \quad (12)$$

第二检测点 $S_2 = (t, L)$ 的代理候选区域取为四个半径 1000 m 圆盘的交，并排除距离端点过近的位置：

$$\Omega_0 = \{(t, L) : 5.1 \leq \|S_2 - G_{d,\sigma}\|_2 \leq 1000, \forall (d, \sigma)\}. \quad (13)$$

式 (??) 对四个极端端点同时施加接收约束与近距离排除约束，使候选点首先满足保守的可行性筛选。由于这里采用端点场景代替连续不确定集， Ω_0 是后续极小极大优化的工程代理域，而不是对所有连续场景的解析充要描述。图 ?? 给出了窄扇形、候选区域和最终最优点。

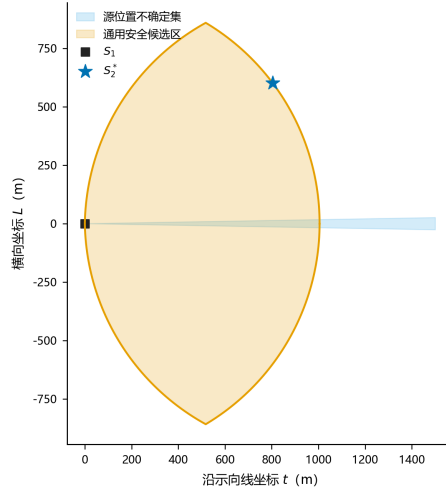


图 2 第一次测向后的源可能集、通用接收候选区与最优第二检测点

5.2.3 精确定位区域与极小极大模型

记干扰源到两个检测点的距离为 d_1, d_2 ，两条真实示向线的交会角为 φ 。当 ε 较小时，两条宽度为 2ε 的方位楔在交点附近形成近似平行四边形，其面积为

$$A(d_1, d_2, \varphi) \approx \frac{4\varepsilon^2 d_1 d_2}{|\sin \varphi|}. \quad (14)$$

式 (??) 表明, 过大的 $d_1 d_2$ 和过小的交会角都会放大定位区域。最终程序还用半平面裁剪直接计算交会多边形面积, 并在随机几何样本上验证该近似式的适用性。令

$$M_A(t, L) = \max_{\substack{d \in \{5, 1500\} \\ \sigma \in \{-1, 1\}}} A(d, \|G_{d, \sigma} - S_2\|_2, \varphi_{d, \sigma}(t, L)), \quad (15)$$

其中 $\varphi_{d, \sigma}(t, L)$ 为从 S_1 与 S_2 指向端点 $G_{d, \sigma}$ 的两条真实连线夹角, 面积函数 $A(\cdot)$ 按式 (??) 计算。由此建立端点代理极小极大模型

$$(t^*, L^*) = \arg \min_{(t, L) \in \Omega_0} M_A(t, L). \quad (16)$$

采用多起点 Nelder–Mead 搜索求解式 (??), 最终得到

$$t^* = 928.7948 \text{ m}, \quad L^* = \pm 382.7989 \text{ m}. \quad (17)$$

因此通用布设策略为

$$S_2^* = S_1 + 928.8 \mathbf{e}_{\parallel} \pm 382.8 \mathbf{e}_{\perp}. \quad (18)$$

正、负 L 两个解关于第一次示向线镜像。四角点代理目标值为 2259.84 m^2 ; 该值用于比较候选点, 不将其解释为连续不确定集上的严格最坏上界。候选区内的目标分布见图 ??。

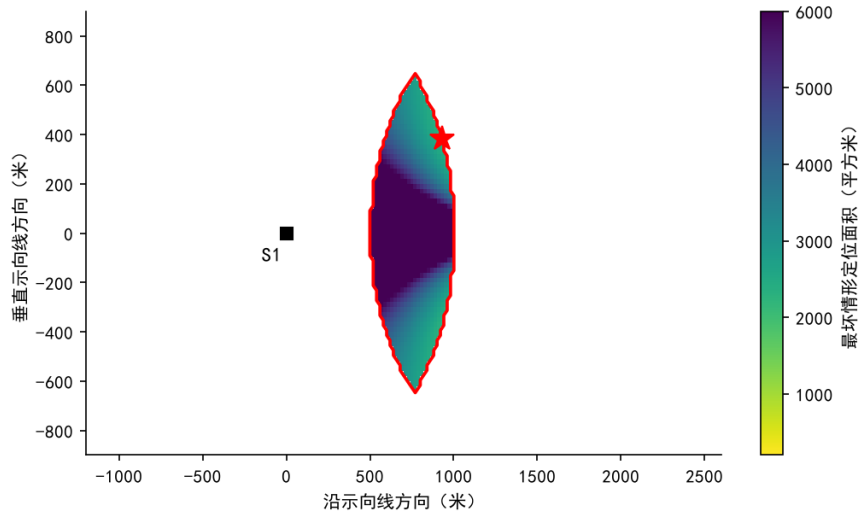


图 3 候选区域内的端点最大定位面积及最优点

5.2.4 蒙特卡洛实验验证

为检验方案在非端点场景下的总体表现, 最终程序生成 3000 个共享随机场景, 让五种策略在同一组干扰源位置上配对比较。 d_1 在 $[50, 1500]$ 上均匀抽样, 偏角 α 在 $[-1^\circ, 1^\circ]$

上均匀抽样；二次接收仍按保守半径 1000 m 判定。表 ?? 的样本数为成功接收且定位区域有界的次数，面积统计也只对这些样本计算。

极小极大策略在 3000 次试验中无接收失败，平均面积与中位面积分别为 691.56 m^2 和 510.95 m^2 ，样本最大值为 2264.57 m^2 。相比之下，沿示向线、垂直偏移和 45° 折中策略均出现接收失败；已知距离策略虽效果更好，但使用了本题第二次布点前并不知道的 d_1 ，只作为理想参照。

表 4 五种第二检测点策略的蒙特卡洛比较

策略	成功样本	失败数	平均面积/ m^2	中位面积/ m^2	标准差/ m^2	最大值/ m^2
沿示向线 1000 m	2406	594	7213.81	8057.16	6751.59	65303.24
垂直偏移 600 m	1517	1483	576.59	458.92	441.31	1636.91
极小极大最优	3000	0	691.56	510.95	457.61	2264.57
45° 折中 700 m	2678	322	895.59	497.93	860.45	3424.20
已知距离最优	3000	0	10.71	7.44	10.52	53.71

5.2.5 模型灵敏度分析

进一步改变第一次接收后可能距离的上界 $d_{\max}$ ，并在每个水平上重新求解式 (??)。图 ?? 显示，随着 $d_{\max}$ 从 300 m 增至 1500 m，最优点沿示向线偏移量总体减小，垂直偏移量和端点代理目标显著增大。这说明距离不确定范围越宽，第二检测点越需要兼顾远端接收约束与交会角质量。

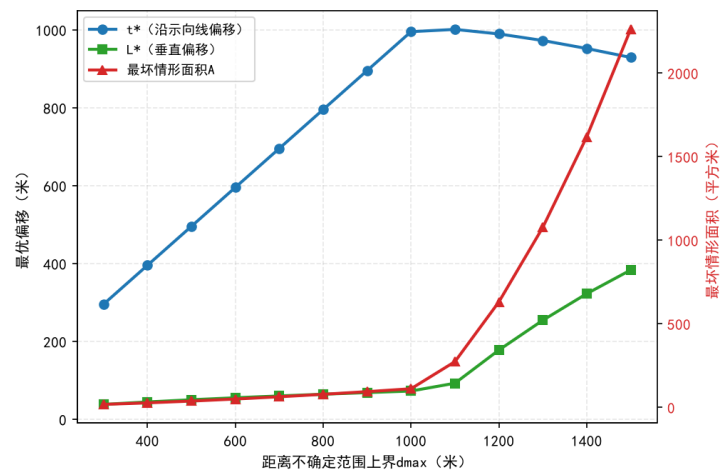


图 4 最优偏移与端点代理目标随 $d_{\max}$ 的变化

综上，在不使用 S_1 绝对位置的条件下，推荐从第一示向线方向前进约 928.8 m，再向任一侧横移约 382.8 m。若实际任务中已知 S_1 的全局坐标，可用目标圆进一步裁剪 W_0 并重新优化，此时正、负 L 方向不再必然等价。

5.3 问题 3：机器狗搜索清除模型——全向干扰源

5.3.1 任务时间核算模型

将机器狗的动作分为移动、检测、频道切换和清除四类。设动作序列为 $\pi = (a_1, a_2, \dots, a_M)$ ，第 j 个动作的位置为 P_j ，并令 $P_0 = (0, 0)$ 。记检测次数、频道切换次数、成功清除次数和失败清除次数分别为 N_{mea} 、 N_{sw} 、 N_{ok} 和 N_{fail} ，则定位清除总时间为

$$T_{\text{total}}(\pi) = \frac{1}{v} \sum_{j=1}^M \|P_j - P_{j-1}\|_2 + 5N_{\text{mea}} + N_{\text{sw}} + 5N_{\text{ok}} + 3N_{\text{fail}}, \quad (19)$$

其中机器狗速度 $v = 5 \text{ m/s}$ 。测向动作固定耗时 5 s ；仅当本次检测频道与当前频道不同时产生 1 s 切换耗时；成功清除和失败清除分别耗时 5 s 和 3 s 。在全部干扰源均被清除的约束下，优化目标为

$$\min_{\pi} T_{\text{total}}(\pi), \quad T_{\text{avg}} = \frac{T_{\text{total}}}{N_c}, \quad (20)$$

其中 N_c 为被清除干扰源数。式 (20) 表明，移动耗时与检测耗时是主要可优化项，因此算法同时压缩访问路径并减少已充分定位频道的后续测量。

5.3.2 覆盖扫描网格

对全向干扰源，只要预设扫描点的半径 R_{scan} 圆盘覆盖整个目标区域，且 $R_{\text{scan}} \leq R_{\text{min}}$ ，即可得到确定性的检测完备性条件

$$\Omega_{\text{arena}} \subseteq \bigcup_i \text{Disk}(P_i, R_{\text{scan}}), \quad R_{\text{scan}} \leq R_{\text{min}} = 1000 \text{ m}. \quad (21)$$

由式 (21)，对任意干扰源 $G \in \Omega_{\text{arena}}$ ，至少存在一个扫描点 P_i 满足 $\|G - P_i\|_2 \leq R_{\text{scan}} \leq R$ ，因此该源必能被检测。圆覆盖理论为此类布点问题提供了经典理论基础 [?]; 本文针对有限圆域采用显式几何构造并再用数值网格复核，而不直接以无限平面的渐近覆盖结论代替有限域证明。

为兼顾安全裕量与探测点数量，取 $R_{\text{scan}} = 950 \text{ m}$ 。先采用 1 个圆心点加 6 个正六边形顶点环的基本结构。设该环半径为 r_{ring} ，边界中点（相邻环点的角平分线上）到最近环点的距离为

$$d_{\text{bd}} = \sqrt{R_{\text{arena}}^2 + r_{\text{ring}}^2 - 2R_{\text{arena}}r_{\text{ring}} \cos 30^\circ}. \quad (22)$$

令 $d_{\text{bd}} \leq R_{\text{scan}}$ ，并要求中心圆盘与外环圆盘之间不产生覆盖空洞，可取

$$r_{\text{ring}} = \frac{2R_{\text{arena}} \cos 30^\circ - \sqrt{(2R_{\text{arena}} \cos 30^\circ)^2 - 4(R_{\text{arena}}^2 - R_{\text{scan}}^2)}}{2} \approx 1254.8 \text{ m}. \quad (23)$$

进一步，原点附近干扰源若只被原点检测，其多线交会角可能较差。因此在原点两侧各 300 m 处增设两个辅助探测点，用于提高近心区域获得第二条有效示向线的概率。最终网格共 9 个探测点。离散高密度校核所得全域最大覆盖距离为 $949.97 \text{ m} < 950 \text{ m}$ ；另在 10 万次随机位置统计中，每个干扰源平均被 2.19 个探测点覆盖，说明该网格不仅满足“至少检测一次”的完备性要求，也为后续交会定位保留了多方位观测冗余。

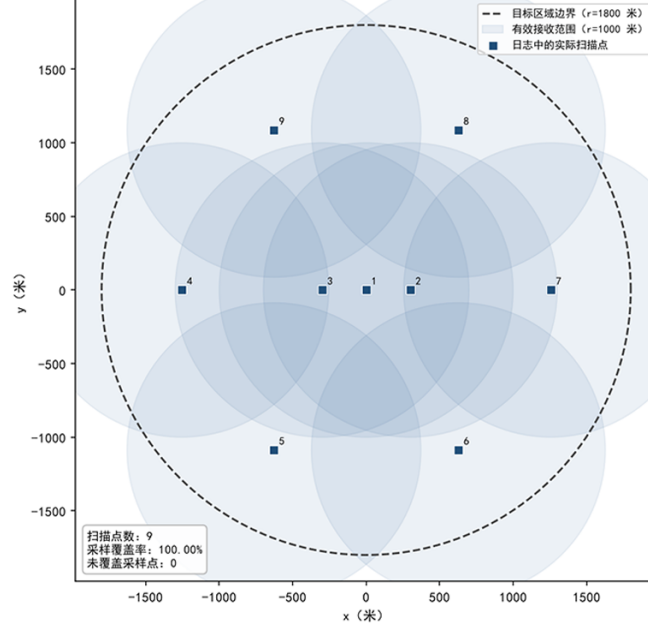


图 5 覆盖扫描网格与完备性验证

5.3.3 交会粗定位与自适应补测

在扫描点 P_i 获得示向度 θ_i 后，以误差上界 $\varepsilon = 1^\circ$ 构造示向扇形

$$\mathcal{W}_i = \{X \in \Omega_{\text{arena}} : |\text{wrap}(\arg(X - P_i) - \theta_i)| \leq \varepsilon\}, \quad \mathcal{P} = \bigcap_i \mathcal{W}_i. \quad (24)$$

由问题 1 的算法，干扰源清除位置估计为定位多边形 $\mathcal{P}$ 的最小包围圆圆心

$$(\hat{G}, \rho) = \arg \min_{z \in \mathbb{R}^2} \max_{x \in \mathcal{P}} \|x - z\|_2, \quad (25)$$

其中 ρ 是最坏位置误差界。当 $\rho \leq 19 \text{ m}$ 时，预留 1 m 数值裕量后可直接实施半径 20 m 的光学清除。

当定位多边形直径 $D_{\mathcal{P}} \leq 200 \text{ m}$ 时，直接进入清除阶段；否则采用受问题 2 “增大交会角、同时控制二次距离” 原则启发的补测：从首条示向线点出发沿与示向线成 45° 的方向移动约 707 m 后再测一次。该补测点并非式 (??) 的极小极大最优点，而是为了减

少移动距离和简化在线执行所采用的折中方案。获得第三条示向线后重新计算交会多边形；若全向源补测无信号，则结合接收半径范围对源距离作进一步排除。

5.3.4 清除执行与两级细定位模型

清除阶段以最近邻生成访问顺序，并可用 2-opt 对开放路径作局部改进 [?]. 到达粗定位中心 $\hat{G}$ 后先直接尝试清除（清除半径为 20 m）；若失败，则在当前位置 Q_1 测得方位 θ_1 ，再沿其垂直方向移动基线 $b = 25$ m 到 Q_2 测得 θ_2 。记 $\mathbf{u}(\theta) = (\cos \theta, \sin \theta)$ ，两条示向直线的交点可写为

$$\tilde{G} = Q_1 + \frac{(Q_2 - Q_1) \times \mathbf{u}(\theta_2)}{\mathbf{u}(\theta_1) \times \mathbf{u}(\theta_2)} \mathbf{u}(\theta_1), \quad (26)$$

其中二维叉积结果为标量。当两条示向线近似平行时，分母接近 0，程序不直接采用该交点，而转入步进补测，以避免数值放大。

在小角度近似下，若两次测向点到目标的典型距离均不超过 d ，则基线为 b 的两点交会误差可保守估计为

$$e_{\text{fine}} \lesssim \frac{2\varepsilon d^2}{b}. \quad (27)$$

当 $d \leq 100$ m、 $b = 25$ m、 $\varepsilon = \pi/180$ 时，式 (27) 给出 $e_{\text{fine}} \lesssim 13.96$ m，小于 20 m 清除半径。该估计说明 25 米基线在典型粗定位误差尺度下具有足够的误差压缩能力，但它是小角度近似下的保守量级分析，并不代替实际交会多边形的数值校核。若交会点距离过远（超过 120 m）或检测无信号，则沿示向线步进并逐点尝试清除，每三步重新测向修正方向，直至清除成功。

5.3.5 算法优势与优化策略

优化算法的优势主要体现在以下方面：

1. **开放路径联合优化。**“最近邻初解 + 2-opt”同时作用于扫描路线和清除路线 [?]，不要求回到起点，减少无效折返。直接针对式 (??) 中占比最大的移动项，且不增加接口调用次数。
2. **以最坏误差驱动清除。**最小包围圆半径 ρ 给出定位区域到清除点的统一上界，贴合 20 m 清除半径这一几何约束，可减少无效清除与后续细定位。

5.3.6 问题 3 测试统计

首先，在官方模拟器上完成 10 次演练测试，统计结果如表 ?? 所示。10 次测试均清除全部干扰源，各次“平均每源时间”的算术均值为 421.9 s，标准差为 65.0 s；若以 10

次总时间除以共清除的 124 个源，则加权平均为 399.74 s。移动耗时均值为 4346.0 s，约占平均总时间 5271.0 s 的 82.5%，说明后续优化应优先压缩扫描与清除路线。

表 5 问题 3 演练测试统计（10 次）

统计指标	最小值	最大值	均值	标准差
被清除干扰源个数比例	1.000	1.000	1.000	0.000
平均定位清除时间 (秒)	287.9	564.7	421.9	65.0
定位清除总时间 (秒)	4215.7	6222.5	5271.0	597.6
移动耗时 (秒)	3292.7	5325.5	4346.0	583.6
检测耗时 (秒)	570.0	825.0	714.3	64.9
频道切换耗时 (秒)	109.0	162.0	139.2	13.4
单源主动清除耗时 (秒)	15.1	707.8	160.7	103.9
指令总数	132	181	158.2	11.6

在 10 次充分演练之后，将按正式测试流程进行 3 次正式测试，结果如表 ?? 所示。

表 6 问题 3 正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间 (秒)	程序运行时间 (秒)
5T4T-Q82P-PGEJ-8K6K	11	430.57	0.46
F75K-SFZA-7QM2-393Z	12	334.00	0.35
P3K6-KUQP-XHUB-RP7D	12	398.67	0.21

图 ?? 显示移动时间是问题 3 测试中的主要耗时来源；图 ?? 展示各次测试的时间变化；图 ?? 则表明扫描点访问、定位与逐源清除能够形成连续闭环。三次正式测试共清除 35 个干扰源，按干扰源数加权的平均每源时间为 386.53 s，最大程序运行时间为 0.46 s，三次均完成全部清除，验证了策略在模拟器环境中的可执行性。

5.4 问题 4：机器狗搜索清除模型——全向 & 定向干扰源

5.4.1 定向干扰源检测条件的建模

设定向干扰源位于 G 、定向方向为 γ （未知），其辐射覆盖角为 $[\gamma - 90^\circ, \gamma + 90^\circ]$ 。检测点 P 能检测到该干扰源，当且仅当距离和方向两个条件同时满足：

$$\|G - P\|_2 \leq R, \quad |\text{wrap}(\angle(GP) - \gamma)| \leq 90^\circ. \quad (28)$$

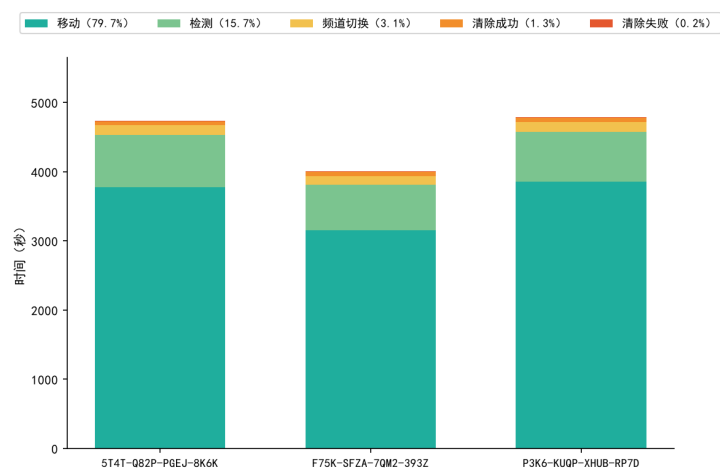


图 6 正式测试时间构成堆积柱状图

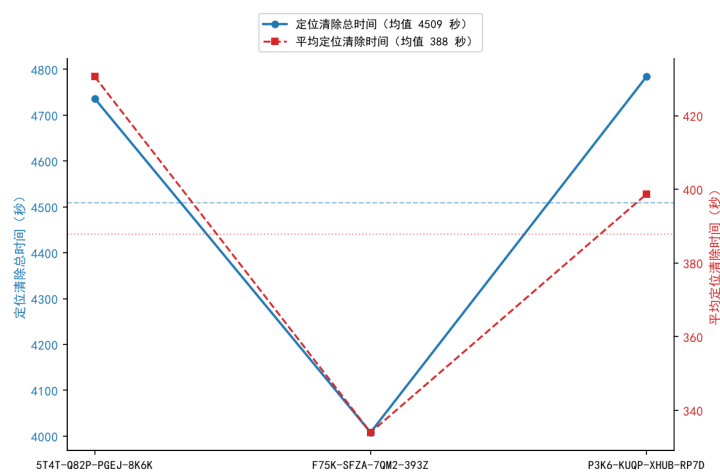


图 7 问题 3 测试时间折线图

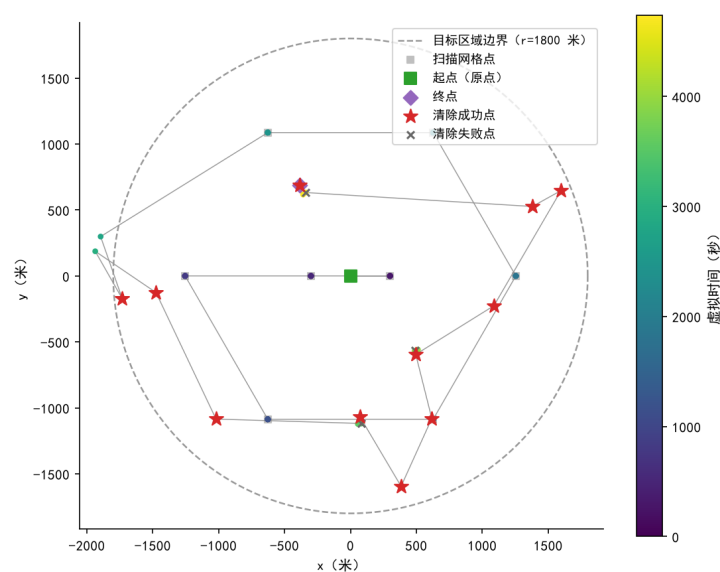


图 8 问题 3 一次测试的完整搜索清除轨迹

其中 $\angle(GP)$ 表示向量 $P - G$ 的方位角。为了对未知 $R \in [1000, 1500]$ m 给出最坏情形保证，只考察半径下界内的距离可达探测点集合

$$\mathcal{S}_R(G) = \{P_i : \|P_i - G\|_2 \leq R_{\min}\}. \quad (29)$$

将 $\mathcal{S}_R(G)$ 中各点相对 G 的方位角按圆周升序排列为 $\beta_1, \dots, \beta_m$ ，并令 $\beta_{m+1} = \beta_1 + 360^\circ$ 。定义最大方向间隙

$$\alpha_{\text{gap}}(G) = \max_{1 \leq i \leq m} (\beta_{i+1} - \beta_i). \quad (30)$$

若 $\alpha_{\text{gap}}(G) \leq 180^\circ$ ，则圆周上任意长度为 180° 的发射半圆都至少包含一个距离可达探测方向，从而对任意未知定向角 γ ，都存在 $P_i \in \mathcal{S}_R(G)$ 满足式 (??)。反之，若最大空缺角大于 180° ，即可把定向源的辐射半圆完全放入该空缺区，使全部距离可达探测点均位于背向区。因此

$$\boxed{\alpha_{\text{gap}}(G) \leq 180^\circ, \quad \forall G \in \Omega_{\text{arena}}} \quad (31)$$

就是本文三环网格的方向完备性判据。该判据把“距离够近”与“方向可见”统一到同一几何条件中。

5.4.2 三环扫描网格的建立与数值验证

网格由三个同心环共 28 个探测点组成：内环 5 点（半径 607.2 米、72 度间隔）、中环 11 点（半径 1374.6 米）、外环 12 点（半径 1984.9 米、30 度间隔，布置于目标区域外）。设计逻辑如下：

(i) 内环与中环（共 16 点）保证区域内部的距离覆盖：中环 11 点按 32.7 度间隔布置，位于相邻中环点角平分线上的边界点到最近中环点的距离为

$$d_{\text{mid}} = \sqrt{1800^2 + 1374.6^2 - 2 \times 1800 \times 1374.6 \times \cos \frac{180^\circ}{11}} = 617.6 \text{ m} < 1000 \text{ m}. \quad (32)$$

区域边界亦被中环覆盖，内环与中环的覆盖圆盘相互重叠，区域内部无空洞；对完整 28 点网格的数值统计显示，区域内任意位置均被至少 3 个、平均 5.17 个探测点覆盖，为交会定位提供充足的多方位信息；

(ii) 内环保证近心干扰源获得多条不同方位的示向线（问题 2 已论证示向点附近的单点定位质量差）；

(iii) 外环的作用是保证“朝外发射”的边界干扰源可检测：边界源距其正前方最近外环点的距离不超过 $\sqrt{1800^2 + 1984.9^2 - 2 \times 1800 \times 1984.9 \times \cos 15^\circ} = 526.9$ 米，且外环点位于区域外、处于该源定向方向的前方覆盖角内，故外环点必能从覆盖角内、有效接收半径内检测到朝外源。三个环的半径与初始相位经数值试验选定。以式 (??) 为判据，在 400 个半径层、900 个方向角、总计 36 万个网格位置上数值检验：全部位置的最

大方向间隙不超过 174.3 度，即

$$\max_{G \in \Omega_{\text{arena}}} \alpha_{\text{gap}}(G) = 174.3^\circ \leq 180^\circ - 5.7^\circ. \quad (33)$$

完备性成立。扫描时按最近邻加 2-opt 微调 [?] 的开放路径巡游 28 个探测点，巡游总长约 21.3 千米。

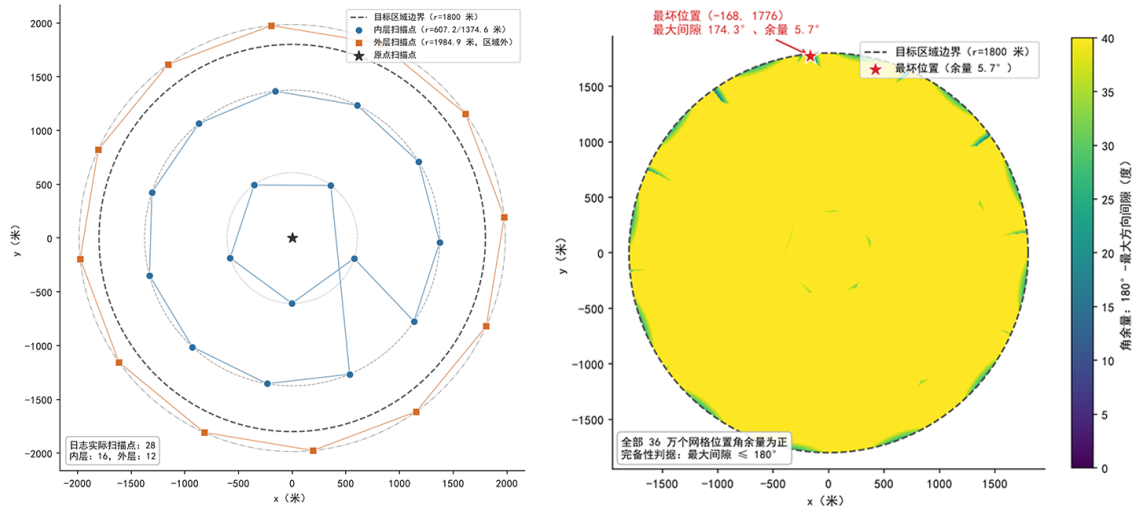


图 9 三环扫描网格布局与完备性验证

5.4.3 扫描剪枝与粗定位模型

逐点逐频道扫描时，若某频道已获得足够定位信息则在该点跳过检测：以交会多边形的最小包围圆 [?] 半径不超过 19 米或已获得不少于 4 条示向线为“充分”判据，前者保证位置估计足以支撑 20 米半径的直接清除（留 1 米裕量），后者避免过度扫描。粗定位沿用问题 1 的半平面交算法，位置估计取交会多边形的最小包围圆圆心，定位误差界取最小包围圆半径。对交会不足的频道，沿问题 3 采用的 45°、约 707m 工程化补测方向增加一次观测；与问题 3 不同的是，补测返回无信号不再判定干扰源位于远方——定向源可能因检测点位于覆盖角外而无法被观测，此时转入单示向线蛇形扇区覆盖（见步骤 4）。

5.4.4 清除执行与兜底模型

清除阶段先按定位中心生成最近邻开放路径，再用 2-opt 进行局部改进 [?]。到达估计点 $\hat{G}$ 后：

(a) 先直接尝试清除——清除成功与否只与距离有关、与覆盖角无关，即便从背面接近定向源亦可直接尝试；

(b) 失败后在当前点检测：返回距离过近则直接清除；返回示向度则以问题 3 的两点细定位（25 米垂直基线）交会出精细位置，交会点不超过 120 米则就地清除，仍失败则沿示向线步进、每 3 步修正一次方向；

(c) 返回无信号（可能从背面接近定向源）时启动环形探测：绕 $\hat{G}$ 以 $r_k = 40k$ 米（ $k = 1, \dots, 4$ ）为半径、每圈 6 个方向（60 度间隔）探测，由于定向源覆盖角为 180 度，任一探测圈上至少 3 个方向必落在覆盖角内，必能获得示向度或距离过近；对估计误差 $\leq 4 \times 40 - 20 = 140$ 米的干扰源，探测圈必能捕获其信号，随后转入两点细定位与步进清除；

(d) 全部失败时以十字加螺旋清除兜底。对仅有一条示向线的频道，执行蛇形扇区覆盖：沿示向线每 30 米设置一个清除点、纵向覆盖 1500 米，横向以 ± 13.2 米偏移布置两条蛇形轨迹。单条示向线在 1500 米处的 1 度误差扇区横向半宽为 $1500 \times \tan 1^\circ \approx 26.2$ 米，两条轨迹结合 20 米清除半径合计覆盖 ± 33.2 米，完全覆盖误差扇区，保证扇区内的干扰源必被清除。

5.4.5 问题 4 测试统计

首先，我们在官方模拟器上完成了 10 次演练测试，统计结果如表 ?? 所示。

10 次演练被清除干扰源个数的比例全部为 100%，验证了三环网格对定向干扰源的检测完备性与清除流程的可靠性；平均定位清除时间均值为 651.4 秒，标准差 134.0 秒，变异系数 0.206，最差情形 905.6 秒。与问题 3 相比，探测点数由 9 增至 28、指令数均值由 158.2 增至 362，移动与检测耗时相应增加，这是覆盖角约束带来的必要代价。10 次演练共清除 130 个干扰源，全部在到达估计点后首次清除即成功，蛇形扇区覆盖、环形探测与螺旋兜底均未触发，说明三环网格提供的多方位交会已使粗定位误差稳定落在 20 米清除半径之内。

表 7 问题 4 演练测试统计汇总（10 次）

统计指标	最小值	最大值	均值	标准差
被清除干扰源个数比例	1.000	1.000	1.000	0.000
平均定位清除时间 (秒)	485.8	905.6	651.4	134.0
定位清除总时间 (秒)	7772.2	9056.2	8241.4	383.6
移动耗时 (秒)	5923.7	6265.2	6086.4	113.9
检测耗时 (秒)	1330.0	2285.0	1742.5	268.4
频道切换耗时 (秒)	265.0	456.0	347.5	53.7

表 7 问题 4 演练测试统计汇总（续）

统计指标	最小值	最大值	均值	标准差
单源主动清除耗时 (秒)	16.4	560.8	146.1	82.8
指令总数	282	467	362	52

在 10 次充分演练之后，将按正式测试流程进行 3 次正式测试，结果如表 ?? 所示。

表 8 问题 4 正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间 (秒)	程序运行时间 (秒)
3SVX-SWPP-TC8V-26QN	16	506.35	0.81
78TS-42MR-AG8H-SRTW	11	731.77	3.35
96ZV-DWNR-C4WM-VD2Q	10	851.82	2.19

图 ?? 显示移动仍是问题 4 测试中的主要耗时来源；图 ?? 展示各次测试的时间变化；图 ?? 表明扫描点访问、定位与逐源清除形成了连续闭环。三次正式测试共清除 37 个干扰源，按干扰源数加权的平均每源时间为 666.74 s，最大程序运行时间为 3.35 s，三次均完成全部清除。相较问题 3，额外时间主要来自对定向源的探测，而不是清除动作本身。

综上，问题 4 以距离可达探测点的最大方向间隙为核心判据设计 28 点三环扫描网格。36 万个位置的离散校核得到最大方向间隙 $174.3^{\circ} < 180^{\circ}$ ；10 次演练清除比例均为 100%，平均定位清除时间为 651.4 s。理论判据与模拟器结果相互印证，说明该策略能够应对定向源从背向探测无信号的困难。

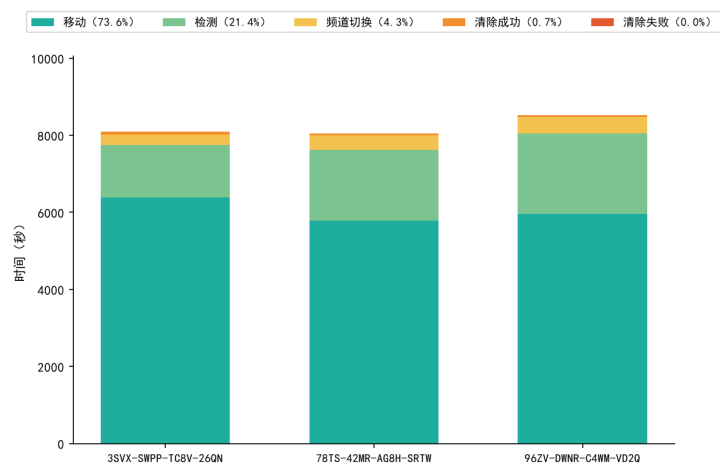


图 10 问题 4 时间构成堆积柱状图

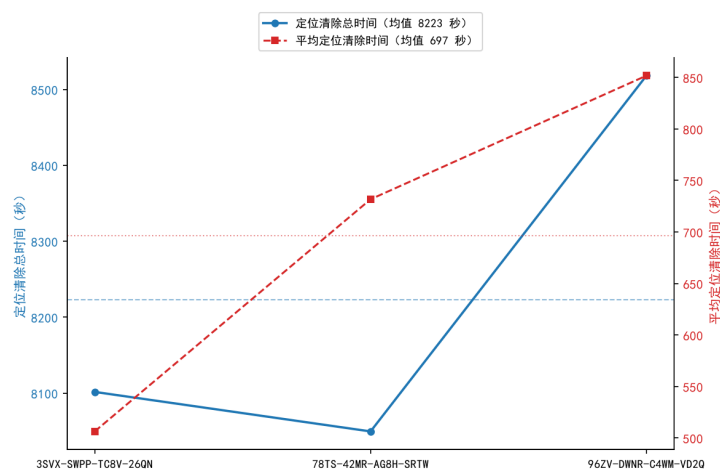


图 11 问题 4 各次测试时间折线图

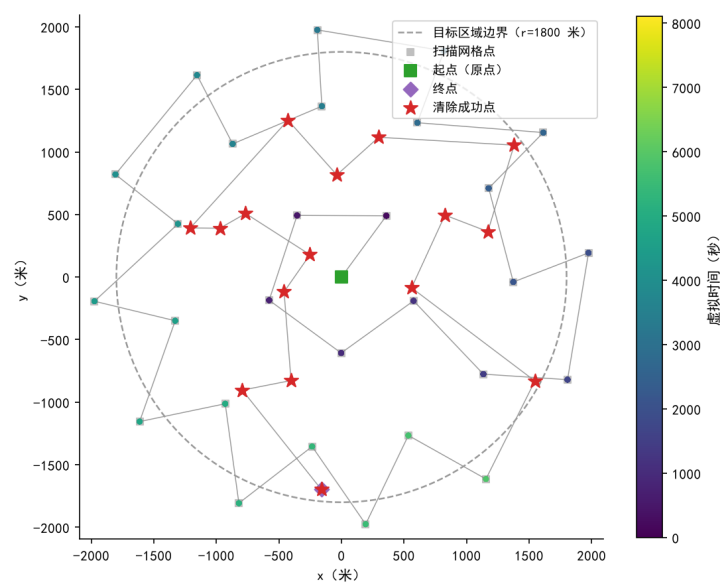


图 12 问题 4 一次测试的完整搜索清除轨迹

六、模型的评价及推广

6.1 模型评价

本文提出的机器狗搜索清除模型具有以下优点：

优点 1 完备性有理论保证。问题 3 与问题 4 的扫描网格分别以 950 米覆盖裕量与最大方向间隙判据设计并经数值验证，从理论上保证任意位置（及任意朝向）的干扰源必被检测，两个问题各 10 次演练测试清除比例全部达到 100%，可靠性得到双重确认。

优点 2 定位精度与清除决策具有统一的几何解释。示向度误差有界使交会定位区域始终包含真值，最小包围圆半径直接给出清除点的最坏误差界；在典型 $d \leq 100\text{m}$ 的细定位尺度下，式 (??) 给出的误差量级约为 14m，小于 20m 清除半径。问题 3、4 演练中的单源主动清除耗时均值分别为 160.7s 与 146.1s，说明定位精度提高后能够减少反复试探。

优点 3 模型层次清晰、可解释性强。问题 1 给出覆盖性问题的严格几何答案（容格定理上下界与反例），问题 2 的极小极大解析框架揭示了“距离与交会角折中”的内在规律，为问题 3、4 的策略设计提供了直接理论依据。

优点 4 工程可实现性好。正式测试中问题 3 与问题 4 的程序运行时间上界分别为 0.46s 与 3.35s，相较机器狗数千秒量级的物理移动与检测时间可以忽略。因此，主要优化矛盾确实集中在路径长度、检测次数和补测策略，而不是计算复杂度。

此外，从鲁棒性角度看，本文尽量把关键保证建立在区间边界而非概率均值上：问题 2 按 $R_{\min}$ 约束二次接收，问题 3 用 950m 裕量半径避免贴近接收边界，问题 4 则只使用 $R_{\min}$ 内的距离可达探测点计算方向间隙。这种设计会牺牲一部分平均效率，但能减少因接收半径未知、定向角未知而导致的漏检风险。

同时，模型也存在一些不足：

缺点 1 覆盖扫描的固定成本较高。为保证完备性，问题 3 需扫描全部 20 个频道、问题 4 需巡游 28 个探测点，当干扰源数量较少时（如 10 个），平均定位清除时间被扫描固定成本显著抬高（问题 4 最坏情形达 905.6 秒）。

缺点 2 策略对有效接收半径下限的依赖较强。覆盖网格以 1000 米为判据设计，若实际环境中存在有效接收半径更小的干扰源，完备性将被破坏，需相应加密网格。

6.2 模型改进与推广

本模型可沿两个方向进一步改进。**其一**，在扫描与清除的耦合优化方面，可将覆盖扫描路线与清除路线联合求解：把“扫描全部探测点”与“清除全部干扰源”视为一个整体的路径规划问题，允许在扫描过程中顺路清除已定位干扰源，从而减少总行程；更进一步，可利用检测结果动态调整后续探测点顺序，对已确认无信号的区域提前终止扫描。**其二**，在参数自适应方面，可根据演练中积累的统计规律动态调整补测触发阈值与

步进步长：例如在检测到交会角普遍良好时放宽补测条件，在多次清除失败时自动缩小步长，使策略在不同电磁环境下自适应。此外，针对个别清除失败场景，可引入多机协同或多次复扫机制进一步提高可靠性。

本模型具有良好的推广价值。在方法层面，扇形交会加半平面交的定位框架适用于一切带角度观测误差的测向定位系统，与电子战中的无源测向定位方法一脉相承 [?]，可推广到声呐目标定位、天文观测定位与基站测角定位等场景；覆盖扫描的完备性设计方法则可推广到无人机的区域侦察路径规划、环境监测传感器布设与灾后搜救路线制定等“带检测半径约束的全覆盖搜索”问题 [?]。在应用层面，本文的机器狗策略可扩展至多机器人协同清除、移动干扰源的追踪清除以及三维空间中的干扰源定位等更复杂的工程场景，为无线电频谱管理的智能化提供算法储备。

七、AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于生成样例测试数据、代码调试、语言润色，详细使用情况见支撑材料。

参考文献

- [1] 王永良, 陈辉, 彭应宁, 等. 空间谱估计理论与算法 [M]. 北京: 清华大学出版社, 2004.
- [2] 赵国庆. 雷达对抗原理 [M]. 2 版. 西安: 西安电子科技大学出版社, 2012.
- [3] SUTHERLAND I E, HODGMAN G W. Reentrant polygon clipping[J]. Communications of the ACM, 1974, 17(1): 32–42.
- [4] TOUSSAINT G T. Solving geometric problems with the rotating calipers[C]//Proceedings of IEEE MELECON’83. Athens: IEEE, 1983: A10.02/1–4.
- [5] WELZL E. Smallest enclosing disks (balls and ellipsoids)[M]//MAURER H. New Results and New Trends in Computer Science. Berlin: Springer, 1991: 359–370.
- [6] JUNG H. Uber die kleinste Kugel, die eine raumliche Figur einschliesst[J]. Journal fur die reine und angewandte Mathematik, 1901, 123: 241–257.
- [7] TOKEKAR P, ISLER V. Sensor placement and selection for bearing sensors with bounded uncertainty[C]//2013 IEEE International Conference on Robotics and Automation. Karlsruhe: IEEE, 2013: 2515–2520.
- [8] KERSHNER R. The number of circles covering a set[J]. American Journal of Mathematics, 1939, 61(3): 665–671.

- [9] CROES G A. A method for solving traveling-salesman problems[J]. Operations Research, 1958, 6(6): 791–812.
- [10] POISEL R A. Electronic warfare target location methods[M]. 2nd ed. Boston: Artech House, 2012.
- [11] CHOSET H, LYNCH K M, HUTCHINSON S, et al. Principles of robot motion: theory, algorithms, and implementations[M]. Cambridge: MIT Press, 2005.

附录 A 问题 3 求解代码–Python 源代码

```
import json
import os
import time
import math
import sys
from urllib.request import Request, urlopen
from urllib.error import URLError, HTTPError

# ----- 配置 -----
BASE_URL = 'http://127.0.0.1:2026'
SPEED = 5.0 # 移动速度(米/秒)
SCAN_R = 950.0 # 覆盖裕量(<1000)
RING_R = 1254.8 # 环形扫描半径
CENTER_OFF = 300.0 # 原点两侧辅助探测点偏移
GOOD_SIN = 0.35 # 交会角质量阈值
LOCALIZATION_QUALITY_THRESHOLD = 200.0 # 定位质量阈值(交会多边形直径, 米)
PROBE_T = 500.0 # 补测点沿示向线偏移
PROBE_L = 500.0 # 补测点垂直偏移

def scan_grid():
    """9个扫描探测点: 原点 + 原点两侧 + 环形6点"""
    pts = [(0.0, 0.0), (CENTER_OFF, 0.0), (-CENTER_OFF, 0.0)]
    for k in range(6):
        a = math.radians(k*60)
        pts.append((RING_R*math.cos(a), RING_R*math.sin(a)))
    return pts

# ----- HTTP 客户端 -----
class SimClient:
    """官方协议客户端: 带重试的POST, 校验accepted"""
    def __init__(self, base_url=BASE_URL, robot_id=None):
        if not robot_id:
            raise ValueError('必须提供参赛队号 robot_id')
        self.base_url = base_url
        self.robot_id = robot_id
        self.req_seq = 0
        self.log = [] # 请求/响应日志
        self.move_t = 0.0 # 移动耗时
        self.switch_t = 0.0 # 频道切换耗时
        self.measure_t = 0.0 # 检测耗时
        self.clear_ok_t = 0.0 # 清除成功耗时
        self.clear_fail_t = 0.0
        self.virtual_time = 0.0
        self.pos = (0.0, 0.0)
```

```

self.channel = 1

def _post(self, path, payload, retries=6):
    self.req_seq += 1
    data = json.dumps(payload).encode('utf-8')
    for att in range(retries):
        try:
            req = Request(self.base_url + path, data=data,
                           headers={'Content-Type': 'application/json'}, method='POST')
            with urlopen(req, timeout=10) as r:
                resp = json.loads(r.read().decode('utf-8'))
                break
        except (URLError, HTTPError, TimeoutError, OSError) as e:
            if isinstance(e, HTTPError):
                try:
                    resp = json.loads(e.read().decode('utf-8'))
                    break
                except Exception:
                    resp = None
            time.sleep(0.2*(att+1))
            resp = None
    else:
        resp = None
    if resp is None:
        raise RuntimeError(f'{path} 请求失败')
    self.log.append(dict(path=path, payload=str(payload), resp=str(resp),
                        virtual_time=resp.get('virtual_time_s', 0)))
    if not resp.get('accepted', False):
        raise RuntimeError(f'{path} 未执行: {resp}')
    self.virtual_time = float(resp['virtual_time_s'])
    return resp

def enter(self):
    r = self._post('/enter', dict(arena_id='default', robot_id=self.robot_id,
                                   request_id=f'e{self.req_seq}'))
    self.remaining_real = r['remaining_real_duration_s']
    return r

def measure(self, x, y, ch):
    move_t = math.hypot(x-self.pos[0], y-self.pos[1])/SPEED
    switch_t = 1.0 if ch != self.channel else 0.0
    r = self._post('/measure', dict(arena_id='default', robot_id=self.robot_id,
                                   request_id=f'm{self.req_seq}',
                                   position={'x': x, 'y': y}, channel=ch))

    self.pos = (x, y)
    self.channel = ch
    self.move_t += move_t

```

```

        self.switch_t += switch_t
        self.measure_t += 5.0
        return r, move_t, switch_t

def clear(self, x, y, ch):
    move_t = math.hypot(x-self.pos[0], y-self.pos[1])/SPEED
    r = self._post('/clear', dict(arena_id='default', robot_id=self.robot_id,
                                request_id=f'c{self.req_seq}',
                                position={'x': x, 'y': y}, channel=ch))

    self.pos = (x, y)
    self.move_t += move_t
    if r.get('clear_result') == 'success':
        self.clear_ok_t += 5.0
    else:
        self.clear_fail_t += 3.0
    return r, move_t

def exit(self):
    r = self._post('/exit', dict(arena_id='default', robot_id=self.robot_id,
                                request_id=f'x{self.req_seq}'))

    return r

# ----- 几何工具 -----
def _wedge_hps(S, theta_deg):
    t = math.radians(theta_deg)
    e = math.radians(1.0)
    u1 = (math.cos(t-e), math.sin(t-e))
    u2 = (math.cos(t+e), math.sin(t+e))
    return [(u1[1], -u1[0], u1[1]*S[0]-u1[0]*S[1]),
            (-u2[1], u2[0], -u2[1]*S[0]+u2[0]*S[1])]

def _clip(poly, hp, eps=1e-9):
    a, b, c = hp
    if not poly:
        return []
    out = []
    n = len(poly)
    for i in range(n):
        P, Q = poly[i], poly[(i+1) % n]
        fP, fQ = a*P[0]+b*P[1]-c, a*Q[0]+b*Q[1]-c
        if fP <= eps:
            out.append(P)
        if (fP <= eps) != (fQ <= eps):
            t = fP/(fP-fQ)
            out.append((P[0]+t*(Q[0]-P[0]), P[1]+t*(Q[1]-P[1])))
    cl = []
    for p in out:

```

```

        if not c1 or math.hypot(p[0]-c1[-1][0], p[1]-c1[-1][1]) > 1e-6:
            c1.append(p)
    if len(c1) > 1 and math.hypot(c1[0][0]-c1[-1][0], c1[0][1]-c1[-1][1]) <= 1e-6:
        c1.pop()
    return c1

def localization_polygon(bearings, cap=2000.0):
    """bearings: [(x, y, svd_deg), ...] -> 交会多边形顶点列表(含区域边界裁剪)"""
    poly = [(-cap, -cap), (cap, -cap), (cap, cap), (-cap, cap)]
    for x, y, th in bearings:
        for hp in _wedge_hps((x, y), th):
            poly = _clip(poly, hp)
            if len(poly) < 3:
                return []
    if len(poly) < 3:
        return []
    return poly

def poly_centroid(verts):
    A = 0.0; cx = 0.0; cy = 0.0
    n = len(verts)
    for i in range(n):
        x1, y1 = verts[i]; x2, y2 = verts[(i+1) % n]
        f = x1*y2 - x2*y1
        A += f; cx += (x1+x2)*f; cy += (y1+y2)*f
    if abs(A) < 1e-12:
        return (sum(p[0] for p in verts)/n, sum(p[1] for p in verts)/n)
    return (cx/(3*A), cy/(3*A))

def poly_diameter(verts):
    best = 0.0
    n = len(verts)
    for i in range(n):
        for j in range(i+1, n):
            d = math.hypot(verts[i][0]-verts[j][0], verts[i][1]-verts[j][1])
            best = max(best, d)
    return best

def best_cross_sin(bearings):
    """两两示向线的交会角正弦最大值(>GOOD_SIN认为交会良好)"""
    best = 0.0
    n = len(bearings)
    for i in range(n):
        for j in range(i+1, n):
            P = _line_intersect(bearings[i], bearings[j])
            if P is None:
                continue

```

```

        a1 = math.atan2(P[1]-bearings[i][1], P[0]-bearings[i][0])
        a2 = math.atan2(P[1]-bearings[j][1], P[0]-bearings[j][0])
        best = max(best, abs(math.sin(a1-a2)))
    return best

def _line_intersect(b1, b2):
    x1, y1, t1 = b1; x2, y2, t2 = b2
    u1 = (math.cos(math.radians(t1)), math.sin(math.radians(t1)))
    u2 = (math.cos(math.radians(t2)), math.sin(math.radians(t2)))
    det = u1[0]*u2[1] - u1[1]*u2[0]
    if abs(det) < 1e-9:
        return None
    dx, dy = x2-x1, y2-y1
    s = (dx*u2[1] - dy*u2[0])/det
    return (x1 + s*u1[0], y1 + s*u1[1])

def nearest_order(points, start):
    """最近邻排序"""
    remain = list(points)
    order = []
    cur = start
    while remain:
        nxt = min(remain, key=lambda p: math.hypot(p[0]-cur[0], p[1]-cur[1]))
        order.append(nxt)
        remain.remove(nxt)
        cur = nxt
    return order

def optimize_open_route(points, start=(0.0, 0.0)):
    """最近邻初始化+2-opt, 优化不要求回到起点的访问路径。"""
    route = nearest_order(list(points), start)
    if len(route) < 3:
        return route
    def length(seq):
        cur, total = start, 0.0
        for p in seq:
            total += math.hypot(p[0]-cur[0], p[1]-cur[1])
            cur = p
        return total
    best = length(route)
    improved = True
    while improved:
        improved = False
        for i in range(len(route)-1):
            for j in range(i+1, len(route)):
                cand = route[:i] + list(reversed(route[i:j+1])) + route[j+1:]
                value = length(cand)

```

```

        if value + 1e-7 < best:
            route, best, improved = cand, value, True
    return route

def minimum_enclosing_circle(points):
    """小规模凸多边形的确定性最小包围圆，返回((x,y), radius)。"""
    pts = list(points)
    if not pts:
        return ((0.0, 0.0), float('inf'))
    def contains(circle, p):
        (cx, cy), r = circle
        return math.hypot(p[0]-cx, p[1]-cy) <= r + 1e-7
    candidates = [(p[0], p[1]), 0.0] for p in pts
    for i, a in enumerate(pts):
        for b in pts[i+1:]:
            c = ((a[0]+b[0])/2.0, (a[1]+b[1])/2.0)
            candidates.append((c, math.hypot(a[0]-b[0], a[1]-b[1])/2.0))
    for i, a in enumerate(pts):
        for j in range(i+1, len(pts)):
            b = pts[j]
            for c in pts[j+1:]:
                d = 2*(a[0]*(b[1]-c[1]) + b[0]*(c[1]-a[1]) + c[0]*(a[1]-b[1]))
                if abs(d) < 1e-10:
                    continue
                ux = ((a[0]**2+a[1]**2)*(b[1]-c[1]) +
                    (b[0]**2+b[1]**2)*(c[1]-a[1]) +
                    (c[0]**2+c[1]**2)*(a[1]-b[1]))/d
                uy = ((a[0]**2+a[1]**2)*(c[0]-b[0]) +
                    (b[0]**2+b[1]**2)*(a[0]-c[0]) +
                    (c[0]**2+c[1]**2)*(b[0]-a[0]))/d
                candidates.append(((ux, uy), math.hypot(ux-a[0], uy-a[1])))
    feasible = [z for z in candidates if all(contains(z, p) for p in pts)]
    return min(feasible, key=lambda z: z[1])

# ----- 策略主体 -----
class StrategyQ3:
    def __init__(self, client):
        self.c = client
        self.trace = dict(actions=[], sources={}) # sources[ch] = 事件记录

    def run(self):
        c = self.c
        self.t0 = 0.0
        c.enter()
        self.cleared = set()
        self.bearings = {} # ch -> [(x, y, svd)]
        self.first_heard = {} # ch -> 虚拟时刻

```

```

self.hear_point = {} # ch -> (x, y)
# ----- 第一阶段：覆盖扫描 -----
# 访问集合不变，只优化开放路径；不影响1000米覆盖完备性。
for idx, (gx, gy) in enumerate(optimize_open_route(scan_grid(), c.pos)):
    act = sorted([ch for ch in range(1, 21) if ch not in self.cleared])
    for ch in act:
        if ch in self.bearings and self._scan_sufficient(self.bearings[ch]):
            continue
        r, _, _ = c.measure(gx, gy, ch)
        self.trace['actions'].append(dict(kind='scan', x=gx, y=gy, ch=ch,
                                           res=r.get('measure_result'),
                                           svd=r.get('svd_deg'), t=c.virtual_time))
        if r['measure_result'] == 'near':
            cr, _ = c.clear(gx, gy, ch)
            self.trace['actions'].append(dict(kind='clear_near', x=gx, y=gy, ch=ch,
                                              res=cr.get('clear_result'), t=c.virtual_time))
            if cr['clear_result'] == 'success':
                self.cleared.add(ch)
                self.trace['sources'][ch] = dict(first_heard=c.virtual_time,
                                                  cleared_t=c.virtual_time, method='near')
                self.first_heard[ch] = c.virtual_time
            elif r['measure_result'] == 'direction':
                self.bearings.setdefault(ch, []).append((gx, gy, r['svd_deg']))
                if ch not in self.first_heard:
                    self.first_heard[ch] = c.virtual_time
                    self.hear_point[ch] = (gx, gy)
# ----- 第二阶段：补测与就地清除(弱交会源，按空间最近邻) -----
weak = []
good = []
for ch in sorted(set(range(1, 21)) - self.cleared):
    if ch not in self.bearings:
        continue
    if self._well_localized(self.bearings[ch]):
        good.append(ch)
    else:
        weak.append(ch)
remain_w = set(weak)
cur = c.pos
while remain_w:
    ch = min(remain_w, key=lambda cc: self._weak_dist(cc, cur))
    remain_w.remove(ch)
    if ch in self.cleared:
        continue
    if self._probe_and_kill(ch):
        cur = c.pos
        continue
    est, bound = self._far_estimate(ch)

```



```

        cur = est
        self._kill(ch, est, bound)
# ----- 第三阶段：良好交会源最近邻清除 -----
estimates = {}
for ch in good:
    poly = localization_polygon(self.bearings[ch])
    if poly:
        estimates[ch] = minimum_enclosing_circle(poly)
point_to_ch = {estimates[ch][0]: ch for ch in estimates}
order = [point_to_ch[p] for p in optimize_open_route(point_to_ch, c.pos)]
for ch in order:
    if ch in self.cleared:
        continue
    est, bound = estimates[ch]
    self._kill(ch, est, bound)
# ----- 第四阶段：完备性核查与退出 -----
for ch in range(1, 21):
    if ch in self.cleared:
        continue
    if ch in self.bearings:
        est, bound = self._far_estimate(ch)
        self._kill(ch, est, bound*2 + 200)
# 未听到任何信号的频道：无干扰源(覆盖扫描完备性保证)
c.exit()
return self._summary()

def _well_localized(self, bearings):
    """交会多边形直径不超过定位质量阈值时，视为定位良好。"""
    if len(bearings) < 2:
        return False
    poly = localization_polygon(bearings)
    if not poly:
        return False
    return poly_diameter(poly) <= LOCALIZATION_QUALITY_THRESHOLD

def _scan_sufficient(self, bearings):
    """达到稳定交会后停止扫描；精度不足交给后续近场补测。"""
    return self._well_localized(bearings)

def _weak_dist(self, ch, cur):
    """弱交会源补测点与当前位置的距离(选靠近当前位置的一侧)"""
    if ch not in self.bearings:
        return 1e12
    x0, y0, th0 = self.bearings[ch][0]
    u = (math.cos(math.radians(th0)), math.sin(math.radians(th0)))
    best = 1e12
    for side in (1, -1):

```

```

        n = (-u[1]*side, u[0]*side)
        px, py = x0 + PROBE_T*u[0] + PROBE_L*n[0], y0 + PROBE_T*u[1] + PROBE_L*n[1]
        best = min(best, math.hypot(px-cur[0], py-cur[1]))
    return best

def _probe_and_kill(self, ch):
    """到补测点补测并就地清除。返回True表示已处理"""
    c = self.c
    x0, y0, th0 = self.bearings[ch][0]
    u = (math.cos(math.radians(th0)), math.sin(math.radians(th0)))
    sides = sorted([1, -1],
                    key=lambda s: math.hypot(x0+PROBE_T*u[0]+PROBE_L*(-u[1]*s)-c.pos[0],
                                              y0+PROBE_T*u[1]+PROBE_L*(u[0]*s)-c.pos[1]))

    est, bound = None, None
    for side in sides:
        n = (-u[1]*side, u[0]*side)
        px, py = x0 + PROBE_T*u[0] + PROBE_L*n[0], y0 + PROBE_T*u[1] + PROBE_L*n[1]
        r, _, _ = c.measure(px, py, ch)
        self.trace['actions'].append(dict(kind='probe', x=px, y=py, ch=ch,
                                          res=r.get('measure_result'),
                                          svd=r.get('svd_deg'), t=c.virtual_time))

        if r['measure_result'] == 'near':
            cr, _ = c.clear(px, py, ch)
            if cr['clear_result'] == 'success':
                self._mark(ch, c.virtual_time, 'probe_near')
                return True
            return False

        if r['measure_result'] == 'direction':
            self.bearings.setdefault(ch, []).append((px, py, r['svd_deg']))
            poly = localization_polygon(self.bearings[ch])
            if poly:
                est = poly_centroid(poly)
                bound = poly_diameter(poly)/2
            break

    if est is None:
        est, bound = self._far_estimate(ch)
    self._kill(ch, est, bound)
    return True

def _far_estimate(self, ch):
    """仅依首条示向线的远方点估计"""
    x0, y0, th0 = self.bearings[ch][0]
    u = (math.cos(math.radians(th0)), math.sin(math.radians(th0)))
    return (x0 + 1433*u[0], y0 + 1433*u[1]), 300.0

def _kill(self, ch, est, bound):
    c = self.c

```

```

t_kill = c.virtual_time
ex, ey = est
# 直接尝试清除
r, _ = c.clear(ex, ey, ch)
self.trace['actions'].append(dict(kind='clear', x=ex, y=ey, ch=ch,
                                   res=r.get('clear_result'), t=c.virtual_time))

if r['clear_result'] == 'success':
    self._mark(ch, t_kill, 'direct')
    return
# 检测当前点示向度
r, _, _ = c.measure(ex, ey, ch)
self.trace['actions'].append(dict(kind='fine_measure', x=ex, y=ey, ch=ch,
                                   res=r.get('measure_result'),
                                   svd=r.get('svd_deg'), t=c.virtual_time))

if r['measure_result'] == 'near':
    cr, _ = c.clear(ex, ey, ch)
    if cr['clear_result'] == 'success':
        self._mark(ch, t_kill, 'near')
        return
if r['measure_result'] == 'direction':
    if self._fine_fix(ch, ex, ey, r['svd_deg'], t_kill):
        return
# 细定位失败：从当前点沿示向线步进
self._walk_kill(ch, (ex, ey), r['svd_deg'], max(bound, 30.0), t_kill)
return
# 无信号：十字+螺旋兜底
self._walk_kill(ch, (ex, ey), None, max(bound, 30.0), t_kill)

def _fine_fix(self, ch, ex, ey, th, t_kill):
    """两点细定位：垂直基线25米交会 -> 交点处清除。成功返回True"""
    c = self.c
    u = (math.cos(math.radians(th)), math.sin(math.radians(th)))
    n = (-u[1], u[0])
    for side in (1, -1):
        p2 = (ex + 25*n[0]*side, ey + 25*n[1]*side)
        r2, _, _ = c.measure(p2[0], p2[1], ch)
        self.trace['actions'].append(dict(kind='fine_measure', x=p2[0], y=p2[1], ch=ch,
                                           res=r2.get('measure_result'),
                                           svd=r2.get('svd_deg'), t=c.virtual_time))
        if r2['measure_result'] == 'near':
            cr, _ = c.clear(p2[0], p2[1], ch)
            if cr['clear_result'] == 'success':
                self._mark(ch, t_kill, 'fine_near')
                return True
        if r2['measure_result'] == 'direction':
            p3 = _line_intersect((ex, ey, th), (p2[0], p2[1], r2['svd_deg']))
            if p3 is not None and math.hypot(p3[0]-ex, p3[1]-ey) <= 120.0:

```

```

        cr, _ = c.clear(p3[0], p3[1], ch)
        self.trace['actions'].append(dict(kind='clear', x=p3[0], y=p3[1], ch=ch,
                                          res=cr.get('clear_result'), t=c.virtual_time))

        if cr['clear_result'] == 'success':
            self._mark(ch, t_kill, 'fine_fix')

            return True

        # 清除失败：以交点为新起点继续
        ex, ey = p3[0], p3[1]
        break
    return False

def _walk_kill(self, ch, start, th, bound, t_kill):
    """沿示向线步进清除(每步尝试清除，定期修正方向)；th=None时十字+螺旋兜底"""
    c = self.c
    ex, ey = start
    if th is None:
        for dx, dy in [(25, 0), (-25, 0), (0, 25), (0, -25), (18, 18), (-18, 18), (18, -18),
                       (-18, -18)]:
            cr, _ = c.clear(ex+dx, ey+dy, ch)
            if cr['clear_result'] == 'success':
                self._mark(ch, t_kill, 'cross')
                return
        for k in range(1, 6):
            rr = 25.0*k
            for ang in range(0, 360, 45):
                cr, _ = c.clear(ex+rr*math.cos(math.radians(ang)),
                                ey+rr*math.sin(math.radians(ang)), ch)
                if cr['clear_result'] == 'success':
                    self._mark(ch, t_kill, 'spiral')
                    return
        self.trace['sources'][ch] = dict(first_heard=self.first_heard.get(ch,
                                c.virtual_time),
                                cleared_t=None, method='FAILED')

        return
    u = (math.cos(math.radians(th)), math.sin(math.radians(th)))
    step = max(15.0, min(40.0, max(bound/4, 15.0)))
    for k in range(1, 18):
        px, py = ex + k*step*u[0], ey + k*step*u[1]
        cr, _ = c.clear(px, py, ch)
        self.trace['actions'].append(dict(kind='walk_clear', x=px, y=py, ch=ch,
                                          res=cr.get('clear_result'), t=c.virtual_time))

        if cr['clear_result'] == 'success':
            self._mark(ch, t_kill, 'walk')
            return
    if k % 3 == 0:
        mr, _, _ = c.measure(px, py, ch)
        self.trace['actions'].append(dict(kind='walk_measure', x=px, y=py, ch=ch,

```

```

        res=mr.get('measure_result'),
        svd=mr.get('svd_deg'), t=c.virtual_time))

    if mr['measure_result'] == 'near':
        cr2, _ = c.clear(px, py, ch)
        if cr2['clear_result'] == 'success':
            self._mark(ch, t_kill, 'walk_near')
            return
        elif mr['measure_result'] == 'direction':
            u = (math.cos(math.radians(mr['svd_deg'])),
                 math.sin(math.radians(mr['svd_deg'])))
            ex, ey = px, py
            k = 0

    self.trace['sources'][ch] = dict(first_heard=self.first_heard.get(ch, c.virtual_time),
                                     cleared_t=None, method='FAILED')

def _mark(self, ch, t_kill, method):
    c = self.c
    self.cleared.add(ch)
    self.trace['sources'][ch] = dict(first_heard=self.first_heard.get(ch, c.virtual_time),
                                     cleared_t=c.virtual_time,
                                     kill_start=t_kill, active_t=c.virtual_time-t_kill,
                                     method=method)

def _summary(self):
    c = self.c
    cleared_n = len([ch for ch, s in self.trace['sources'].items() if s['cleared_t'] is not
                     None])
    return dict(cleared=cleared_n,
               total_time=c.virtual_time,
               move_t=c.move_t, switch_t=c.switch_t, measure_t=c.measure_t,
               clear_ok_t=c.clear_ok_t, clear_fail_t=c.clear_fail_t,
               n_actions=len(self.trace['actions']),
               sources=self.trace['sources'])

if __name__ == '__main__':
    import argparse
    program_start = time.perf_counter()
    parser = argparse.ArgumentParser(description='机器狗问题3策略')
    parser.add_argument('--robot-id', required=True, help='参赛队号')
    parser.add_argument('--base-url', default=BASE_URL, help=f'模拟器地址(默认: {BASE_URL})')
    args = parser.parse_args()
    client = SimClient(base_url=args.base_url, robot_id=args.robot_id)
    strat = StrategyQ3(client)
    s = strat.run()
    print(json.dumps(s, ensure_ascii=False, indent=2))
    elapsed = time.perf_counter() - program_start
    print(f'整段程序运行时间: {elapsed:.3f} 秒')

```

```

# ---- 自动保存完整日志(含动作轨迹)，供绘图程序使用；保存失败不影响测试 ----
try:
    s['actions'] = strat.trace['actions']
    s['program_run_time_s'] = elapsed
    # 问题三日志独立保存到图表程序默认读取的 logs_q3 目录。
    out_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'logs_q3')
    os.makedirs(out_dir, exist_ok=True)
    out_path = os.path.join(out_dir,
        f'Q3_{args.robot_id}_{time.strftime("%Y%m%d_%H%M%S")}.json')
    with open(out_path, 'w', encoding='utf-8') as f:
        json.dump(s, f, ensure_ascii=False, indent=2)
    print(f'完整日志已保存: {out_path}')
except Exception as e:
    print(f'完整日志保存失败(不影响测试): {e}')

```

附录 B 问题 4 求解代码—Python 源代码

```

import math
import time
import os
import json
from Q3_RobotDog import (SimClient, StrategyQ3, localization_polygon, poly_centroid,
    poly_diameter, best_cross_sin, GOOD_SIN, PROBE_T, PROBE_L)

BASE_URL = 'http://127.0.0.1:2026'

OUTER_R = 2050.0    # 外圈半径（区域外）
OUTER_N = 12        # 外圈点数（30°间隔）
SURVEY_RING_RADIUS = 40.0 # 环形探测基础半径（米）

def optimize_open_route(points, start=(0.0, 0.0)):
    """为一组二维点生成较短的开放路径（最近邻 + 2-opt微调）。

    输入可以是点列表或以点为键的字典；返回点列表，不闭合回起点。
    """
    pts = list(points)
    if not pts:
        return []
    # 先用最近邻得到稳定初始解
    remain = pts[:]
    route = []
    cur = start
    while remain:
        nxt = min(remain, key=lambda p: math.hypot(p[0]-cur[0], p[1]-cur[1]))

```

```

    route.append(nxt)
    remain.remove(nxt)
    cur = nxt
# 开放路径2-opt: 只接受严格改进, 规模很小时开销可忽略
improved = True
while improved:
    improved = False
    for i in range(len(route)-1):
        a = start if i == 0 else route[i-1]
        b = route[i]
        for j in range(i+1, len(route)):
            c = route[j]
            d = route[j+1] if j+1 < len(route) else None
            old = math.hypot(a[0]-b[0], a[1]-b[1])
            if d is not None:
                old += math.hypot(c[0]-d[0], c[1]-d[1])
            new = math.hypot(a[0]-c[0], a[1]-c[1])
            if d is not None:
                new += math.hypot(b[0]-d[0], b[1]-d[1])
            if new + 1e-9 < old:
                route[i:j+1] = reversed(route[i:j+1])
                improved = True
                break
        if improved:
            break
    return route

def minimum_enclosing_circle(points):
    """返回覆盖所有点的最小圆 (中心, 半径)"""
    pts = list(points)
    if not pts:
        return (0.0, 0.0), 0.0
    if len(pts) == 1:
        return pts[0], 0.0

    def circle3(a, b, c):
        ax, ay = a; bx, by = b; cx, cy = c
        d = 2.0 * (ax*(by-cy) + bx*(cy-ay) + cx*(ay-by))
        if abs(d) < 1e-12:
            return None
        aa, bb, cc = ax*ax+ay*ay, bx*bx+by*by, cx*cx+cy*cy
        ux = (aa*(by-cy) + bb*(cy-ay) + cc*(ay-by)) / d
        uy = (aa*(cx-bx) + bb*(ax-cx) + cc*(bx-ax)) / d
        o = (ux, uy)
        return o, math.hypot(ux-ax, uy-ay)

```

```

best = None
# 单点和两点定义的候选圆
for p in pts:
    cand = (p, 0.0)
    if all(math.hypot(q[0]-p[0], q[1]-p[1]) <= 1e-9 for q in pts):
        best = cand if best is None or cand[1] < best[1] else best
for i in range(len(pts)):
    for j in range(i+1, len(pts)):
        o = ((pts[i][0]+pts[j][0])/2, (pts[i][1]+pts[j][1])/2)
        r = math.hypot(pts[i][0]-pts[j][0], pts[i][1]-pts[j][1])/2
        if all(math.hypot(q[0]-o[0], q[1]-o[1]) <= r+1e-9 for q in pts):
            if best is None or r < best[1]:
                best = (o, r)
for i in range(len(pts)):
    for j in range(i+1, len(pts)):
        for k in range(j+1, len(pts)):
            cand = circle3(pts[i], pts[j], pts[k])
            if cand and all(math.hypot(q[0]-cand[0][0], q[1]-cand[0][1]) <= cand[1]+1e-9 for
                q in pts):
                if best is None or cand[1] < best[1]:
                    best = cand
return best if best is not None else (poly_centroid(pts), poly_diameter(pts)/2)

def scan_grid_q4():
    """28点三环覆盖网格，兼顾覆盖完备性与移动距离。"""
    pts = []
    for n, radius, offset_deg in ((5, 607.17512794, 53.75693290),
                                   (11, 1374.62274481, 31.08579950),
                                   (12, 1984.90077183, 5.58974051)):
        for k in range(n):
            a = math.radians(offset_deg + 360.0*k/n)
            pts.append((radius*math.cos(a), radius*math.sin(a)))
    return pts

class StrategyQ4(StrategyQ3):
    def __init__(self, client):
        super().__init__(client)

    def _scan_sufficient(self, bearings):
        if len(bearings) < 2:
            return False
        poly = localization_polygon(bearings)
        if not poly:
            return False
        _, radius = minimum_enclosing_circle(poly)
        return radius <= 19.0 or len(bearings) >= 4

```



```

def run(self):
    c = self.c
    c.enter()
    self.cleared = set()
    self.bearings = {}
    self.first_heard = {}
    # ----- 第一阶段：双层覆盖扫描 -----
    for idx, (gx, gy) in enumerate(optimize_open_route(scan_grid_q4(), c.pos)):
        act = sorted([ch for ch in range(1, 21) if ch not in self.cleared])
        for ch in act:
            if ch in self.bearings and self._scan_sufficient(self.bearings[ch]):
                continue
            r, _, _ = c.measure(gx, gy, ch)
            self.trace['actions'].append(dict(kind='scan', x=gx, y=gy, ch=ch,
                                                res=r.get('measure_result'),
                                                svd=r.get('svd_deg'), t=c.virtual_time))
            if r['measure_result'] == 'near':
                cr, _ = c.clear(gx, gy, ch)
                if cr['clear_result'] == 'success':
                    self.cleared.add(ch)
                    self.trace['sources'][ch] = dict(first_heard=c.virtual_time,
                                                    cleared_t=c.virtual_time, method='near')
                    self.first_heard[ch] = c.virtual_time
            elif r['measure_result'] == 'direction':
                self.bearings.setdefault(ch, []).append((gx, gy, r['svd_deg']))
                if ch not in self.first_heard:
                    self.first_heard[ch] = c.virtual_time
    # ----- 第二阶段：补测与就地清除（弱交会源，按空间最近邻） -----
    weak = []
    good = []
    for ch in sorted(set(range(1, 21)) - self.cleared):
        if ch not in self.bearings:
            continue
        if self._well_localized(self.bearings[ch]):
            good.append(ch)
        else:
            weak.append(ch)
    remain_w = set(weak)
    cur = c.pos
    while remain_w:
        ch = min(remain_w, key=lambda cc: self._weak_dist(cc, cur))
        remain_w.remove(ch)
        if ch in self.cleared:
            continue
        if self._probe_and_kill_q4(ch):
            cur = c.pos
            continue

```

```

        est, bound = self._far_estimate(ch)
        cur = est
        self._kill_q4(ch, est, bound)
# ----- 第三阶段：良好交会源最近邻清除 -----
estimates = {}
for ch in good:
    poly = localization_polygon(self.bearings[ch])
    if poly:
        estimates[ch] = minimum_enclosing_circle(poly)
# 以定位中心规划开放路径；同坐标的不同频道也要全部保留
point_to_ch = {}
for ch, (point, _) in estimates.items():
    point_to_ch.setdefault(point, []).append(ch)
route = optimize_open_route(list(point_to_ch.keys()), c.pos)
order = []
for point in route:
    order.extend(point_to_ch[point])
for ch in order:
    if ch in self.cleared:
        continue
    est, bound = estimates[ch]
    self._kill_q4(ch, est, bound)
# ----- 第四阶段：完备性核查与退出 -----
for ch in range(1, 21):
    if ch in self.cleared:
        continue
    if ch in self.bearings:
        est, bound = estimates.get(ch, self._far_estimate(ch))
        self._kill_q4(ch, est, bound*2 + 200)
c.exit()
return self._summary()

def _probe_and_kill_q4(self, ch):
    """到补测点补测并就地清除（问题4：无信号不代表远方，兜底交_kill_q4）"""
    c = self.c
    x0, y0, th0 = self.bearings[ch][0]
    u = (math.cos(math.radians(th0)), math.sin(math.radians(th0)))
    sides = sorted([1, -1],
                    key=lambda s: math.hypot(x0+PROBE_T*u[0]+PROBE_L*(-u[1]*s)-c.pos[0],
                                              y0+PROBE_T*u[1]+PROBE_L*(u[0]*s)-c.pos[1]))
    est, bound = None, None
    for side in sides:
        n = (-u[1]*side, u[0]*side)
        px, py = x0 + PROBE_T*u[0] + PROBE_L*n[0], y0 + PROBE_T*u[1] + PROBE_L*n[1]
        r, _, _ = c.measure(px, py, ch)
        self.trace['actions'].append(dict(kind='probe', x=px, y=py, ch=ch,
                                           res=r.get('measure_result'),

```

```

        svd=r.get('svd_deg'), t=c.virtual_time))

    if r['measure_result'] == 'near':
        cr, _ = c.clear(px, py, ch)
        if cr['clear_result'] == 'success':
            self._mark(ch, c.virtual_time, 'probe_near')
            return True
        return False
    if r['measure_result'] == 'direction':
        self.bearings.setdefault(ch, []).append((px, py, r['svd_deg']))
        poly = localization_polygon(self.bearings[ch])
        if poly:
            est, bound = minimum_enclosing_circle(poly)
            break
    if est is None:
        return self._fallback_clear_bearing_sector(ch)
    self._kill_q4(ch, est, bound)
    return ch in self.cleared

def _fallback_clear_bearing_sector(self, ch):
    """单导向线时用两条蛇形轨迹覆盖误差扇区，保证定向源可清除。"""
    c = self.c
    x0, y0, th0 = self.bearings[ch][0]
    u = (math.cos(math.radians(th0)), math.sin(math.radians(th0)))
    n = (-u[1], u[0])
    t_kill = c.virtual_time
    longitudinal = [30.0*k for k in range(51)]
    for row, lateral in enumerate((-13.2, 13.2)):
        ts = longitudinal if row == 0 else list(reversed(longitudinal))
        for t in ts:
            px, py = x0 + t*u[0] + lateral*n[0], y0 + t*u[1] + lateral*n[1]
            cr, _ = c.clear(px, py, ch)
            self.trace['actions'].append(dict(kind='sector_clear', x=px, y=py, ch=ch,
                                                res=cr.get('clear_result'), t=c.virtual_time))
            if cr['clear_result'] == 'success':
                self._mark(ch, t_kill, 'sector_cover')
                return True
    self.trace['sources'][ch] = dict(first_heard=self.first_heard.get(ch, c.virtual_time),
                                      cleared_t=None, method='FAILED')
    return False

def _kill_q4(self, ch, est, bound):
    """问题4清除：直接清除失败后，若检测无信号则环形探测找示向线"""
    c = self.c
    t_kill = c.virtual_time
    ex, ey = est
    r, _ = c.clear(ex, ey, ch)
    self.trace['actions'].append(dict(kind='clear', x=ex, y=ey, ch=ch,

```

```

        res=r.get('clear_result'), t=c.virtual_time))

if r['clear_result'] == 'success':
    self._mark(ch, t_kill, 'direct')
    return
# 可能从背面接近：先测；无信号则环形探测
r, _, _ = c.measure(ex, ey, ch)
self.trace['actions'].append(dict(kind='fine_measure', x=ex, y=ey, ch=ch,
        res=r.get('measure_result'),
        svd=r.get('svd_deg'), t=c.virtual_time))

if r['measure_result'] == 'near':
    cr, _ = c.clear(ex, ey, ch)
    if cr['clear_result'] == 'success':
        self._mark(ch, t_kill, 'near')
        return
if r['measure_result'] == 'direction':
    if self._fine_fix(ch, ex, ey, r['svd_deg'], t_kill):
        return
# 细定位失败：从当前点沿示向线步进
self._walk_kill(ch, (ex, ey), r['svd_deg'], max(bound, 30.0), t_kill)
return
# 环形探测：绕估计点按基础半径逐圈、6方向找信号
for k in range(1, 5):
    rr = SURVEY_RING_RADIUS*k
    for ang in range(0, 360, 60):
        px, py = ex + rr*math.cos(math.radians(ang)), ey + rr*math.sin(math.radians(ang))
        mr, _, _ = c.measure(px, py, ch)
        self.trace['actions'].append(dict(kind='survey', x=px, y=py, ch=ch,
            res=mr.get('measure_result'),
            svd=mr.get('svd_deg'), t=c.virtual_time))

        if mr['measure_result'] == 'near':
            cr, _ = c.clear(px, py, ch)
            if cr['clear_result'] == 'success':
                self._mark(ch, t_kill, 'survey_near')
                return
        elif mr['measure_result'] == 'direction':
            if self._fine_fix(ch, px, py, mr['svd_deg'], t_kill):
                return
            self._walk_kill(ch, (px, py), mr['svd_deg'], max(bound, 100.0), t_kill)
            if ch in self.cleared:
                return
# 十字+螺旋清除兜底
self._walk_kill(ch, (ex, ey), None, bound, t_kill)

if __name__ == '__main__':
    import json
    import argparse
    program_start = time.perf_counter()

```

```

parser = argparse.ArgumentParser(description='机器狗问题4策略')
parser.add_argument('--robot-id', required=True, help='参赛队号')
parser.add_argument('--base-url', default=BASE_URL, help=f'模拟器地址（默认：{BASE_URL}）')
args = parser.parse_args()
client = SimClient(base_url=args.base_url, robot_id=args.robot_id)
strat = StrategyQ4(client)
s = strat.run()
print(json.dumps(s, ensure_ascii=False, indent=2))
elapsed = time.perf_counter() - program_start
print(f'完整程序运行时间：{elapsed:.3f} 秒')
# ---- 自动保存完整日志（含动作轨迹），供绘图程序使用；保存失败不影响测试 ----
try:
    import os
    s['actions'] = strat.trace['actions']
    s['program_run_time_s'] = elapsed
    # 问题四日志独立保存到图表程序默认读取的 logs_q4 目录。
    out_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'logs_q4')
    os.makedirs(out_dir, exist_ok=True)
    out_path = os.path.join(out_dir,
        f'Q4_{args.robot_id}_{time.strftime("%Y%m%d_%H%M%S")}.json')
    with open(out_path, 'w', encoding='utf-8') as f:
        json.dump(s, f, ensure_ascii=False, indent=2)
    print(f'完整日志已保存：{out_path}')
except Exception as e:
    print(f'完整日志保存失败（不影响测试）：{e}')

```